

UNIVERSITI TEKNOLOGI MARA

**AI-BASED LOG AGGREGATION
TECHNIQUES FOR SIEM**

AFRINA SYUHADA BINTI JEFFRI SEM

**BACHELOR OF COMPUTER SCIENCE (HONS.)
COMPUTER NETWORKS**

JULY 2026

Universiti Teknologi MARA

**AI-Based Log Aggregation Techniques
for SIEM**

Afrina Syuhada Binti Jeffri Sem

**Thesis submitted in fulfilment of the requirements
for Bachelor of Computer Science (Hons.)
Computer Networks**

Faculty of Computer and Mathematical Sciences

July 2026

SUPERVISOR APPROVAL

MACHINE LEARNING-BASED LOG AGGREGATION FRAMEWORK FOR SIEM SYSTEM

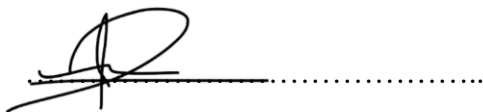
By

AFRINA SYUHADA BINTI JEFFRI SEM

2023800856

This thesis was prepared under the supervision of the project supervisor, Ts. Dr. Mohsen Bin Mohamad Hata. It was submitted to the faculty of Computer and Mathematical Sciences and was accepted in partial fulfilment of the requirements for the degree of Bachelor of Computer Science (Hons.) Computer Networks.

Approved by

A handwritten signature in black ink, consisting of a stylized 'M' and 'H' followed by a horizontal line with a dotted tail.

Ts. Dr. Mohsen Bin Mohamad Hata

Project Supervisor

JULY 2026

STUDENT DECLARATION

I certify that this thesis and the project to which it refers is the product of my own work and that any idea or quotation from the work of other people, published or otherwise are fully acknowledge in accordance with the standard referring practices of the discipline.



.....

Afrina Syuhada Binti Jeffri Sem

2023800856

JULY 2026

ACKNOWLEDGEMENT

By the grace of Allah, the most generous, the most merciful. The strength, the perseverance and the guidance provided by Allah S.W.T gave me the strength to do this Final Year Project successfully.

I should also state that I am the most thankful to my supervisor, Ts. Dr. Mohsen Bin Mohamad Hata, who invaluable supervised me, provided valuable feedback, and sustained me in the course of this project. His knowledge and dedication have been necessary in influencing me to develop the quality and academic depth of my work.

I would also wish to express my gratefulness to lecturers and faculty members of Universiti Teknologi MARA (UiTM) Shah Alam on their part in offering me academic advice and support throughout my period of study. Their patriotism to knowledge and excellence has been of great inspiration to me.

I owe a lot of debt to my family and friends who have been with me all the way, understanding, and encouraging in this venture. The patience and encouragement have been the strength in the difficult time of this journey. To conclude, I would like to appreciate that all of those people have been involved in the successful accomplishment of this project, either directly or indirectly. Your help and cooperation is highly appreciated.

ABSTRACT

While Security Information and Event Management (SIEM) systems are becoming the go-to solution for central security monitoring, they are limited by the volume, variety, and lack of structure of log data generated from today's network, cloud, IoT, and mobile environments. Traditional techniques of logging rely on rules, which demand a lot of manual effort and fail to scale to heterogeneous log formats, leaving inconsistent normalisations, high false-positive rates and significant workload for analysts.

The project proposes and applies an AI-based log aggregation framework for SIEM that consolidates three distinct log sources, Network Firewall log, Android endpoint log and Internet of Things (IoT) traffic under a distributed denial of service (DDoS) attack into uniform standardised log data set of 94,162 events. Unstructured log messages are then transformed into numerical features with Natural Language Processing (NLP) and Term Frequency-Inverse Document Frequency (TF-IDF) vectorisation, and these features are then aggregated with numeric and categorical structured features to get a single, unified representation of the features. This combined feature set is then used to train a supervised Random Forest classifier, which is used to automatically classify attack events as attack or normal, and to give a continuous risk score for each log entry, which is interpretable.

The proposed framework is tested on a held-out test split and validated using various functional testing scenarios in both the log aggregation pipeline and an interactive Security Operations Centre (SOC) dashboard. The trained model can accurately classify attack and normal traffic with 79.81% accuracy and an ROC-AUC score of 89.76%, where the feature importance analysis shows that both textural and structured log attributes are significant for traffic classification. The results show that using NLP-based text representation and an interpretable ensemble classifier is a promising scalable solution for AI-based log aggregation that eliminates the manual processing time and allows for unified, more consistent, and cohesive security monitoring from heterogeneous log sources from an analyst's perspective.

TABLE OF CONTENTS

CONTENT	PAGE
SUPERVISOR APPROVAL	3
STUDENT DECLARATION	4
ACKNOWLEDGEMENT	5
ABSTRACT	6
LIST OF FIGURES.....	10
LIST OF TABLES.....	12
LIST OF ABBREVIATIONS	13
CHAPTER 1	14
1.1 Background of Study	14
1.2 Problem Statement.....	15
1.3 Project Objectives.....	17
1.4 Scope of Project.....	17
1.5 Significant of Project	18
CHAPTER 2	19
2.1 Cybersecurity and Security Monitoring	19
2.1.1 Overview of Cybersecurity Threats	19
2.1.2 Cyberattacks Detection through Log Indicators	20
2.1.3 Security Monitoring in Incident Detection	21
2.2 Security Information and Event Management (SIEM) Systems	22
2.2.1 Purpose of SIEM	22
2.2.2 SIEM Architecture.....	23
2.2.3 SIEM Workflow and Data Processing	25
2.2.4 Limitations of Rule-Based SIEM.....	26
2.3 Log Collection, Heterogeneity and Normalization Challenges	27
2.3.1 Log Collection in SIEM Environments	27
2.3.2 Log Preprocessing and Parsing.....	28
2.3.3 Log Normalization and Aggregation Challenges.....	29
2.4 Machine Learning in Cybersecurity and Log Analysis	30
2.4.1 Overview of Machine Learning Approaches in SIEM.....	30
2.4.2 Supervised vs Unsupervised Learning for Logs	30

2.4.3	Challenges of Applying ML to Log Data.....	32
2.5	NLP for Log Text Representation	32
2.5.1	Textual Nature of Log Messages	32
2.5.2	NLP Preprocessing Techniques	34
2.5.3	TF-IDF and Feature Representation Methods	34
2.5.4	NLP Challenges in Log Analysis.....	35
2.6	Ensemble Learning and Random Forest for Classification.....	35
2.6.1	Random Forest Fundamentals	35
2.6.2	Random Forest in Intrusion and Log Detection.....	36
2.6.3	Handling Class Imbalance with Ensemble Methods.....	37
2.7	Feature Engineering for Heterogeneous Log Attributes.....	37
2.8	Integration of AI-Based Log Analysis into SIEM	38
2.8.1	AI-Enhanced SIEM Architectures	38
2.8.2	Benefits of AI-Based SIEM Systems.....	38
2.9	Related Works.....	39
2.9	Summary of Related Works.....	41
CHAPTER 3		42
3.1	Introduction.....	42
3.2	Project Methodology.....	43
3.2.1	Agile Methodology.....	43
3.3	Requirements Phase	45
3.4	Design Phase.....	47
3.4.1	Software Requirements.....	47
3.4.2	Hardware Requirements	49
3.4.3	Flowchart Diagram.....	51
3.4.4	Entity Relation Diagram (ERD)	57
3.5	Development Phase.....	59
3.6	Testing Phase	60
3.7	Deployment Phase.....	61
3.8	Review Phase.....	62
3.9	Project Milestones.....	63
3.10	Summary.....	64
CHAPTER 4		65
4.1	Introduction.....	65
4.2	Data Description	65
4.2.1	Firewall Log Source	67

4.2.2	Android Log Source	69
4.2.3	IoT Log Source	72
4.3	Testing Scenarios	74
4.4	Results.....	75
4.4.1	Model Evaluation Results (Held-Out Test Set).....	75
4.4.2	Feature Importance	77
4.4.3	Full-Dataset Scoring Results	78
4.4.4	SIEM Dashboard	78
4.5	Discussion.....	85
4.6	Findings	87
CHAPTER 5		89
5.1	Conclusion	89
5.2	Limitation of the Project	90
5.3	Recommendation	92
REFERENCES		94

LIST OF FIGURES

FIGURE	PAGE
2.1 Literature Review Overview	20
2.2 SIEM Dashboard	24
2.3 SIEM Architecture	25
2.4 SIEM Process Flow	26
2.5 Supervised Learning Workflow	32
2.6 Unsupervised Learning Workflow	32
2.7 Natural Language Processing Workflow	34
2.8 Random Forest Classifier	37
3.1 Agile Methodology	44
3.2 Main Flowchart	52
3.3 Log Preprocessing Flowchart	53
3.4 NLP Feature Extraction Flowchart	55
3.5 Feature Engineering Flowchart	56
3.6 Random Forest Classification Flowchart	57
3.7 Entity Relationship Diagram	59
3.8 Gantt Chart	64
4.1 Firewall Severity Distribution	69
4.2 Firewall Risk Score Distribution by AI Prediction	69
4.3 Firewall Top Actions	70
4.4 Android Severity Distribution	71
4.5 Android Risk Score Distribution by AI Prediction	72

4.6	Android Top Event Categories (Components)	72
4.7	IoT Severity Distribution	74
4.8	IoT Risk Score Distribution by AI Prediction	74
4.9	IoT Top Attack Among Predicted Attacks	74
4.10	Random Forest Confusion Matrix on the Held-Out Test Set	77
4.11	ROC curve for the Random Forest classifier (Attack Class)	77
4.12	Top 20 Random Forest feature importances	78
4.13	SIEM Dashboard Login Page	81
4.14	SIEM Dashboard	81
4.15	Overview Tab	82
4.16	Threat Analysis Tab	82
4.17	Defense Tab	82
4.18	Suggested Immediate Response Actions on Critical Event	83
4.19	Incident Response Tab	83
4.20	Firewall Monitoring Tab	83
4.21	Android Monitoring Tab	84
4.22	IoT Monitoring Tab	84
4.23	SOC Report Tab	84
4.24	Explorer Tab	85
4.25	SIEM Dashboard with Sidebar Filter Controls	85

LIST OF TABLES

TABLE	PAGE
2.1 Comparison of Related Works	40
3.1 Details of Agile Methodology	45
3.2 Software Requirements	49
3.3 Hardware Requirements	50
3.4 Entity Relationship Diagram Details	60
4.1 Event count by log source in the unified dataset	67
4.2 Ground-truth label distribution across unified dataset	67
4.3 Severity distribution across the unified dataset	67
4.4 Per-source prediction and risk score summary	68
4.5 Firewall log source summary statistics	69
4.6 Android log source summary statistics	71
4.7 IoT log source summary statistics	73
4.8 Model testing scenarios executed against the implemented system	75
4.9 Random Forest evaluation results on the 20% held-out test split	76
4.10 Top 10 Random Forest feature importances	78
4.11 Full-dataset prediction summary versus held-out test accuracy	79
4.12 SIEM dashboard tabs and their key features	80
4.13 SIEM dashboard sidebar filter controls	85

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
APT	Advanced Persistent Threats
CEE	Common Event Expression
DDoS	Distributed Denial of Service
IoT	Internet of Things
IT	Information Technology
LLMs	Large Language Models
ML	Machine Learning
NLP	Natural Language Processing
ROC-AUC	Receiver Operating Characteristic-Area Under the Curve
SIEM	Security Information and Event Management
TF-IDF	Term Frequency-Inverse Document

CHAPTER 1

INTRODUCTION

In this chapter, a brief explanation regarding project background and other related components for detailing project proposed will be explain.

1.1 Background of Study

Due to the fast digitalisation of businesses, the amount and size of the log data generated by the IT systems today have significantly increased. Logs are collected from servers, network devices, cloud computing, security or mobile apps to record the activities of a system and any potential security issues. As the business infrastructure expands and the use of various technologies increases, the amount of log data is growing exponentially and manual analysis becomes more and more difficult. Generated logs are one of the tools vital in the identification of problems in operations, monitoring security events and maintaining overall cybersecurity position of the organization.

In order to deal with the cybersecurity issues that involve dealing with hundreds and thousands of logs, several organizations apply Security Information and Event Management (SIEM) systems. SIEM systems enhance the visibility in the IT environment of an organization as they centralize the log data collection, normalization, correlation and alerting (Mokalled et al., 2020). However, with respect to cyber threats that are often diverse and dynamic, little can be offered by the existing SIEM system. It can result in alert fatigue and decreased productivity of security analysts and thus, make it harder to locate new threats, false positive rates and to actively update.

Artificial Intelligence (AI) can be used to enhance SIEM capabilities. Machine Learning as a subset of AI technology can be used to analyze vast amounts of data, uncover patterns, and identify unusual behavior, even without

using traditional rule-based approaches (Nurusheva et al., 2024). The use of fixed correlation rules could be cut down, the detection accuracy of threats could be improved and the zero-day attacks could be revealed by implementing AI-driven log analysis in the SIEM environments. However, to be successful in putting Machine Learning to work, clean, relevant and well-structured log data are needed. Most existing methods for aggregating the logs in SIEMs lack advanced features such as automatic feature extraction, removal of noisy data, classification of the logs or semantic enrichment.

The difference between the actual log collection and the AI-driven security analytics could be seen as a demand for a dedicated and intelligent log aggregation solution. This type of framework would integrate logs of various sources to enhance the SIEM scalability to alleviate the exhaustion of security analysts to have several environments under a single system. In the real SIEM environment, the framework will be implemented and the organizations will be able to respond to incidents faster and improve their capability to detect incidents correctly and their overall cybersecurity stance (Hata et al., 2023).

Therefore, the following research will be used for designing and developing AI Based Log Aggregation Techniques and will be implemented on SIEM systems. The recommended approaches will try to improve the ability to aggregate logs by using Artificial Intelligence (AI). Finally, the study will aim at addressing the existing gap in SIEM operations, offering a smarter, scalable and proactive security monitoring, in various settings.

1.2 Problem Statement

Today's cybersecurity environments produce a tremendous amount of log information gathered from a wide variety of sources such as network hardware, cloud services, endpoints or IoT devices. The format, structure and semantics of these logs can greatly differ and using a conventional Security Information and Event Management (SIEM) system to aggregate, normalize and correlate events efficiently at scale is difficult. Traditional SIEM designs focus primarily on fixed rule engines to interpret logs and match event

correlations that can't effectively process high velocity, heterogeneous streams of data in most cases resulting in inconsistent log normalisation, slow log processing and high rate of false positive events that overburden security analysts. The fact that even the conventional accuracy of parsing logs is found to have no better performance when used in downstream analysis shows that, in general, non-adaptive preprocessing techniques for anomaly detection and correlation tasks have limitations (Khan et al., 2024).

According to recent studies, machine learning (ML) and artificial intelligence (AI) solutions can play a crucial role in improving cybersecurity analytics by learning patterns on log data, homogenizing log formats with heterogeneous logs, and minimize false alerts. Some of the examples include the ability to provide more sophisticated AI models, such as Large Language Models (LLM), to improve scalable classification and context awareness of cybersecurity logs, compared to regular ML classifiers (Palma et al., 2025). The use of artificial intelligence (AI) and natural language processing (NLP) in embedding and mapping security events across cloud environments to normalize logs and extract threat signals from a similar source has been found to be promising (Saminathan et al., 2022). Nevertheless, the literature mostly focuses on independent processes like anomaly detection, the creation of synthetic data to be used in classification, or ontology-based log representation instead of a single, complete system that automates the process of aggregating, normalizing and cross-source correlating logs as required by SIEM systems.

Despite those achievements, the existing SIEM products that are available in the market lack scalable and automatic AI-based aggregation capabilities that can aggregate different log sources as unified datasets and can adapt to new formats and semantics. Without these capabilities, SIEM deployment will continue to be constrained by low performance, duplicated alerting, and low correlated event interpretation capabilities, thus denying the ability to perform large-scale security operations effectively. This makes it critical to develop AI-driven log aggregation techniques which can automatically transform a multitude of different log sources into a single normalized data format, remove redundant log data and add higher-level semantic log data to the aggregated data to forward this to the detection and

analysis stages. The proposed project will fill this gap by creating and testing AI-driven methods of log aggregation that enhance the SIEM scalability, minimize redundancy, and enhance the visibility of security events that are later critically evaluated on both benchmark and real-world settings.

1.3 Project Objectives

The objectives of this project are as follows:

- a) To design log aggregation techniques for SIEM using AI-based approaches.
- b) To develop an AI-Based Log Aggregation Techniques for SIEM that unifies diverse log formats from multiple sources into one dataset.
- c) To evaluate the effectiveness of AI-Based Log Aggregation Techniques in improving SIEM systems.

1.4 Scope of Project

The aim of this project is to create AI-Based Log Aggregation Techniques, which will enhance the current Security Information and Event Management (SIEM) systems in terms of efficiency and scalability. This technique entails collecting logs in three different and disparate environments: network firewalls, Android endpoints on mobile devices and the Internet of Things (IoT) devices exposed to DDos traffic. They are mapped on to a common, single, unified schema for SIEM, with fields like 'timestamp', 'source IP', 'destination IP', 'protocol', 'severity' and 'event message' being consistent across all sources.

Each log entry is processed and the free-text component is vectorised to numerical features using Natural Language Processing (NLP), in this case Term Frequency-Inverse Document Frequency (TF-IDF), to make the message

content suitable for the input to a model. The text-based features are merged with structured numeric features (severity score, anomaly score, etc.) and one-hot encoded categorical features (source type, protocol, device type, etc.) to create a single feature matrix. This feature matrix is then used to train the supervised Random Forest classifier to classify an attack event as an attack and a normal event as a normal event, and to generate a continuous ML-based risk score for each log entry. This standardises a wide variety of logs to a single AI scored source, which helps make it easier to monitor security incidents across various environments and improve the scalability and prioritisation of the SIEM system.

1.5 Significant of Project

The importance of this research is that it improves the efficiency, scalability and intelligence of log management in the modern cybersecurity environment. By incorporating AI into the log aggregation process, the suggested techniques can avoid over-flagging as would occur with a traditional rules-based approach and help to save time in manual analysis. By merging logs from network devices, IoT devices, and mobile application into one single analytical platform, it provides visibility across many environments, making it possible for improved security monitoring as well as faster incident response.

Furthermore, this project will facilitate research into new directions in AI-based SIEM design that explicitly explore methods of aggregating logs across multiple sources. It also provides valuable utility, in that it provides a flexible, customisable and adaptive solution that can be extended to a real enterprise network. The results of this project could influence the creation of future SIEM systems based on automation and artificial intelligence to achieve effective cyber security defense, as companies extend their digital presence by integrating connected IoT and mobile devices.

CHAPTER 2

LITERATURE REVIEW

This chapter summarizes the current literature on SIEM systems, log aggregation methods and machine learning log analysis. It discusses the shortcomings of the conventional SIEM solutions for aggregating large and diverse log data and proposes the use of NLP and machine learning clustering methods to enrich log aggregation and correlation. The talk emphasizes the existing problems and research gaps that drive the need for an AI-driven log aggregation approach for SIEM systems.

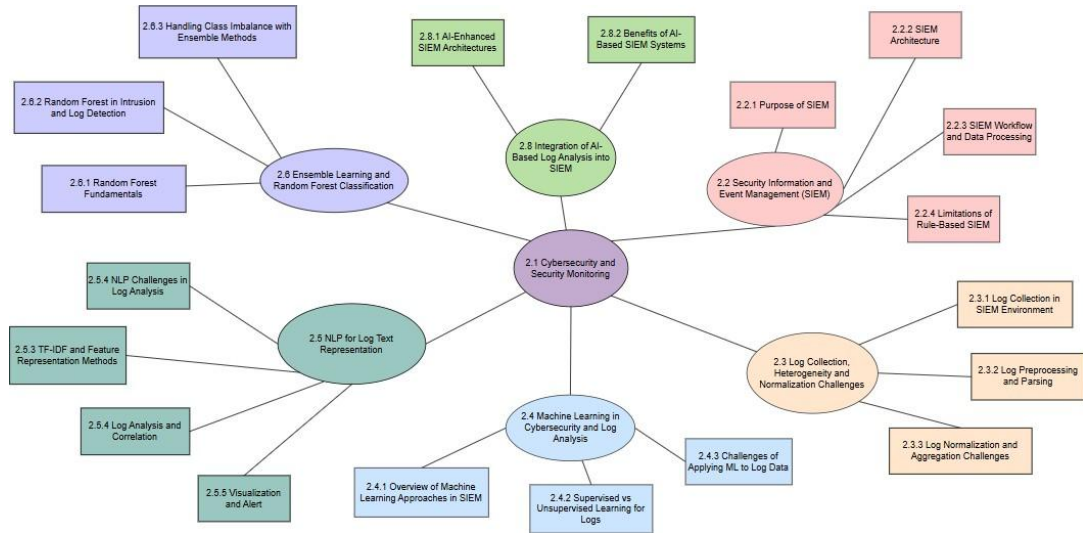


Figure 2.1 Literature Review Overview

2.1 Cybersecurity and Security Monitoring

2.1.1 Overview of Cybersecurity Threats

Threats encompass a broad category of malicious activities that could affect digital assets, networks and systems. All of this has resulted in a fast-

changing modern threat landscape. A comprehensive study of cybersecurity research trends reveals that all these factors, such as automation, artificial intelligence (AI), and new technologies, have led to a rise in the number, complexity, and effects of threats (Takahashi et al., 2025).

Threat actors take advantage of weaknesses in a variety of settings, such as critical infrastructure, IoT systems, and enterprise networks. Although malware, ransomware, phishing, and distributed denial-of-service (DDoS) attacks are still common, new threats have surfaced that use AI and machine learning to automate and scale attacks, making them more difficult to identify with conventional defenses (Alanezi & AL-Azzawi, 2024).

Advanced Persistent Threats (APTs) are another significant type of cyber threat, characterized by prolonged and targeted intrusions. APTs can be dangerous to high-value targets such as government networks and critical infrastructure due to their stealth, persistence, and ability to evade detection methods (Che Mat et al., 2024).

Emerging vectors like AI-driven attacks and vulnerabilities associated with new technologies for computing have been identified in addition to standard and advanced threats. For example, in Neuromorphic mimicry attacks, threat models tailored to next-generation software and hardware architectures become more critical, as they exploit the probabilistic nature of brain-inspired computing systems (Ravipati, 2025).

All these emerging threats create a need for intelligent processing and real-time correlation of a multitude of different security events within the cybersecurity system. The number and diversity of logs from modern infrastructures make automated analysis approaches essential to increase the accuracy of threat detection and improve the efficiency of operations.

2.1.2 Cyberattacks Detection through Log Indicators

Malware, ransomware, phishing, and advanced persistent threats (APTs) are just a few of the many ways that cybersecurity threats can appear.

Security systems need to be able to detect the distinct patterns in logs generated by these threats. The study found that the most exploited vectors include phishing and malware, both of which have a considerable negative effect on an organization's operation and finances (Waliullah et al., 2025). These threats generate a large amount of log information including unauthorized login attempts, abnormal network activity which can provide early warning signs for compromise. Moreover, with the addition of IoT devices and remote services, the attack surface has expanded, and the threat vector is now more complex and spread out, making it hard to successfully interpret with current signature-based detection systems (Rahmati, 2025). With the rise of sophisticated threats, the demand for more sophisticated monitoring and analysis methods has grown, which goes beyond pattern matching and towards models that are driven by data and behaviour and are able to derive meaningful information from security logs.

2.1.3 Security Monitoring in Incident Detection

Effective cybersecurity monitoring is crucial to detect and response to emerging threats on time. Security monitoring requires continuous collection and analysis of event data, to be able to detect threats and anomalies that may be a sign of suspicious activity (Frontiers, 2025). The approach is different from traditional periodic scanning, which could miss out on attacks that are new or irregular. Organizations can detect early indicators of attack and shorten the time that threats remain in systems by regularly analyzing logs and network events (Frontiers, 2025). Furthermore, research has shown that machine learning algorithms can enhance the accuracy of detection when used alongside automated monitoring systems, as they are able to identify patterns in large and complex datasets that human monitoring would not be able to detect (Danish, 2024). These skills are crucial for nowadays security operations, particularly as threat actors use automation and adaptive strategies to get around static defenses.

2.2 Security Information and Event Management (SIEM) Systems

2.2.1 Purpose of SIEM

Security Information and Event Management (SIEM) means a comprehensive security solution that combines real time operation monitoring of Security Event Management (SEM) with the history reporting power of Security Information Management (SIM). The goal of a SIEM system is to provide the organization with a consistent, one-pane view of its security posture by consolidating and integrating the telemetry and log data from many network assets, which are often incompatible with each other, says Gonzalez-Granadillo et al. (2021). Apart from the centralisation of data, the SIEM systems are also designed to transform raw data and high velocity data into actionable intelligence, detect any unusual occurrences, and automate the process of responding to incidents. Manzoor et al. (2025) found that SIEM is no longer being used simply for compliance purposes, but has been replaced by the current Security Operations Center (SOC)'s big data processing machine. In this role, it becomes the most critical infrastructure for identifying and tracking the emergence of sophisticated cyberthreats that traverse multiple network segments to progress laterally and enable security analysts to categorize attacks by environment, risk level (and not simply volume).

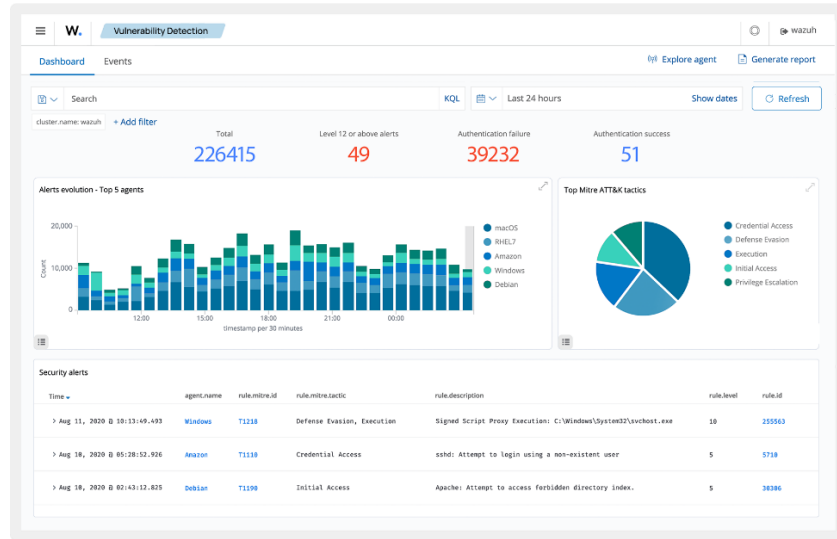


Figure 2.2 SIEM Dashboard

The figure 2.2 above demonstrates Security Information and Event Management (SIEM) dashboard which presents a centralized perspective of security events amassed through various sources of logs. Major security performances such as number of events, severity of alerts, and anomalies identified, are displayed on the dashboard in near real time. The SIEM dashboard enables security analysts to visualize correlated events and trends and thus respond to these effectively and efficiently, helping them to identify suspicious activities and assess potential threats.

2.2.2 SIEM Architecture

There is a multi-level architecture that takes care of a log event's lifetime from its creation to the final destination and hence the structural integrity of a SIEM system is required. Distributed agents and collectors deliver telemetry to SIEM packages in the case of servers, firewalls, databases and cloud services (Sheeraz et al., 2023). This data is then preprocessed and normalized to cope with a variety of formats to allow the cross-source analysis and effective correlation (Sheeraz et al., 2023; Ulhaq et al., 2023). The key element of SIEM is the correlation engines, which can link normalized events

to generate more sophisticated information, detect relationships and patterns between events (Sheeraz et al., 2023; Kummarapurugu, 2024). In recent studies, other architectural changes, such as threat intelligence and analytics layers are also explained, which helps to improve the accuracy of detection and the response speed in hybrid environments (Kummarapurugu, 2024). The support of these layers is backed by the high-performance databases that support quick querying, retention, and retrieval which are imperative both in real-time security operations as well as in forensic investigations.

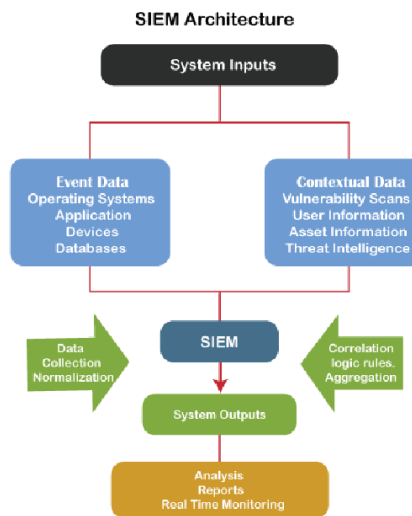


Figure 2.3 SIEM Architecture (Jhaveri and Parmar, 2023)

The explanation of a Security Information and Event Management (SIEM) system general architecture is shown in Figure 2.3 and demonstrates the flow of security information through the key elements of the system. Data sources that support event data that the SIEM can capture are Operating Systems, applications, devices, databases, contextual data such as vulnerability scans, user data, asset data and threat intelligence feeds. Data collection and normalization of these inputs is done to provide consistency of data among heterogeneous sources. The SIEM then applies correlation rules and aggregation logic to events to give them an analytical whole which allows the detection of security incidents and anomalies. The refined outputs of the results are provided as outputs of the system and these can be beneficial in helping the security teams to make decisions and act on incidents in a timely manner to facilitate the functions of security analysis, reporting and real-time monitoring.

2.2.3 SIEM Workflow and Data Processing

The strict rule-matching along with organized data processing are core to the linear, sequential pipeline which forms the typical SIEM operation with the log collection phase being succeeded by deterministic field extraction and correlation. Traditional models of SIEM include collecting the logs passively with the listeners or polling actively, matching fields of the logs via a regular expression or similarity pattern-matching, and then correlating the data (Sheeraz et al., 2023). In the correlation stage, the system uses a predefined series of if-then rules on the normalized event stream to correlate associated activities, as an example, a series of failed login attempts with a successful login of a strange IP address within a set time period would result in a particular alert. This rule-based approach reduces noise and is useful for detecting known sequences of attacks, but it is limited by the need for manual rule writing and maintenance, and can't detect attacks that aren't covered by the pre-defined rules without significant updates (RuleGenie research, Shukla et al., 2025). In fact, according to recent research, many of the most common SIEM detection rules can be trivially defeated by an adversary by simply changing their attack methodology, highlighting that the classical rule-based system is reactive and that alongside deterministic reasoning (rules), it is necessary to deploy adaptive or data-driven approaches to detection (Uetz et al., 2023).

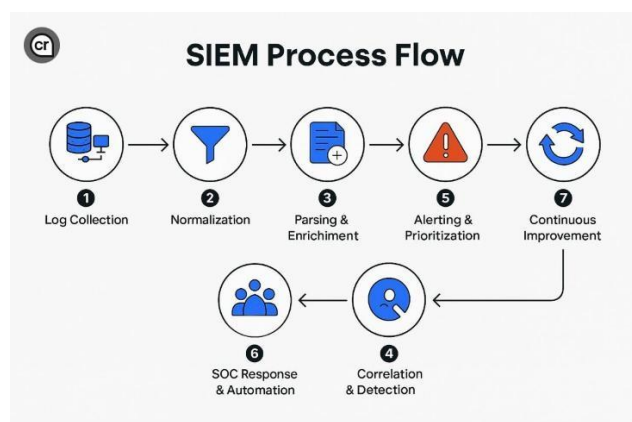


Figure 2.4 SIEM Process Flow

The end-to-end process flow of a Security Information and Event Management (SIEM) system is shown in the figure 2.4. It starts with the stage of collecting logs in which the security events are obtained through various sources. These logs are then normalized to a common format and parsing and enrichment to add contextual information like user, asset, and threat intelligence information is then performed. The SIEM uses the methods of correlation and detection to detect the suspicious patterns and other possible security incidents. There is a level of severity and risk that alerts can be created and prioritized. The detected incidents are handled with SOC Response/Automation which provides for taking timely mitigation action. Lastly, continuous improvement makes sure that the detection rules and response strategies will be improved over time to increase the overall security effectiveness.

2.2.4 Limitations of Rule-Based SIEM

Common SIEM architectures have a number of architectural and operational drawbacks that limit its utility in dealing with modern-day cyber threats. Also, conventional SIEMs rely on inflexible rule-of-thumb correlation engines, with analysts manually writing the rules for known attack signatures, a setup that is inherently unable to recognize new or evolving threats such as zero-day exploits or multi-staged, multi-layer attacks (Shukla et al., 2025). It is found that redundancy and overlap of rules do not only not generalize to unknown attack patterns, but also increase the number of false alerts that leads to alert fatigue and overloads security analysts (Shukla et al., 2025; Olabiyi, 2023). Apart from this, traditional SIEMs struggle to scale in volume, velocity, and variety to process the security data generated in the current cloud-native and hybrid environments, resulting in processing delays and resource overload as the number of logs increases (Kumar et al., 2024).

Since such systems are often under-contextualized and fail to provide adaptive analytics, security teams often receive too many low-value alerts, which may also desensitize them when responding to a critical incident and

lead to neglecting critical incidents (Olabiyi, 2023). All these limitations, with strict rule dependency, high false positive rate, and scalability issues all point to immediate need of more intelligent and adaptive SIEM models which will use machine learning and automated correlation to reduce false positive rates, improve detection rates, and overall threat response efficiency (Shukla et al., 2025; Kumar et al., 2024).

2.3 Log Collection, Heterogeneity and Normalization Challenges

2.3.1 Log Collection in SIEM Environments

The collecting of logs is one of the most fundamental processes of a Security Information and Event Management (SIEM) system because it is important to consolidate and interpret security event data available from many points across an organisation's infrastructure. SIEM platforms will get access to log data on the network devices, servers, endpoints, firewalls and applications and this gathered data is what provides the foundation to real-time detection of threats, incident responses and compliance reports (Heino, 2025). An effective log collection can have the following benefits: It can give the event with the capability for higher level analysis, such as correlating the event and detecting anomalies in the data; It can provide a complete situational awareness, enabling capturing of activity in a distributed environment (Heino, 2025; Adetunmbi and Owolabi, 2021).

According to the research, the SIEM system would not be able to correlate the events and identify subtle security patterns in a large enterprise or cloud-native environment in high volumes and data heterogeneity if there are no effective log ingestion and normalization mechanisms used (Adetunmbi and Owolabi, 2021; Sheeraz et al., 2023). The centralisation of the logs can simplify the management and increase the analytical capability by ensuring that the heterogeneous sources of logs are collected in a central repository, thus improving visibility and capability of investigating the incident and conducting forensic analysis (Sheeraz et al., 2023). As the volume, velocity and variety of

log data keep increasing with the proliferation of cloud service adoption and distributed architectures, the development of scalable and efficient approaches to collecting logs will continue to be vital to maintaining an effective, proactive cybersecurity stance (Adetunmbi and Owolabi, 2021; Heino, 2025).

2.3.2 Log Preprocessing and Parsing

Log pre-processing and parsing are key steps that transform the unstructured raw logs into structured data that can be easily processed by a Security Information and Event Management (SIEM) system. Most of the preprocessing steps involve data cleansing to remove noise and correct inconsistencies in data, and data normalization to ensure consistency in the format of logs despite their diversity in sources (Hajamydeen et al., 2024; Qin et al., 2025). These measures remarkably enhance the quality of data and the computational load during querying and analysis, thus allowing to faster and with higher accuracy investigate security events (He et al., 2024).

Further processing done by log parsing further refines the processed log to useful pieces of data such as time stamps, IP addresses, event identifiers and message templates to help identify behavioral patterns and anomalies in security events. Recent research results show that the correct parsing and structured representation of logs is directly linked to the performance of downstream applications like anomaly detection and threat classification models (He et al., 2024; Piet et al., 2025). It has been demonstrated that advanced parsing methods, such as semantics-based and machine learning-based ones, can decrease ambiguity in the interpretation of logs and improve the accuracy of automated analysis of security (Zhao et al., 2025). Therefore, as far as the data is concerned, preprocessing and parsing will help to maintain its quality and structure when entering SIEM systems, thus improving the quality and reliability of the processes of threat detection.

2.3.3 Log Normalization and Aggregation Challenges

Normalization and consolidation of log data of various and heterogeneous sources have the major challenges that directly influence the efficacy of Security Information and Event Management (SIEM) systems. Log information from network devices, servers, applications, and security tools can be inconsistent (Chuah et al., 2025; Piet et al., 2025), because they may be incompatible, have different structures, syntax, and semantic meaning, making further analysis and detection of threats challenging. Log normalization is a process that converts these different formats into a uniform structured format that can be easily interpreted and analysed across sources, a key precondition for real-time correlation and incident detection (Piet et al., 2025).

The presence of real time processing requirements also extend these issues further – to be able to detect threats in time, SIEM systems must be able to normalize and correlate high velocity log streams with low latency response requirements. It can be challenging to map event fields, event time and context attributes of two or more distinct sources, and processing can be a burden and slow down the system (Tendikov et al., 2024). Additionally, according to industry research, many security teams struggle to scale data normalization, impacting the performance and visibility of their searches in distributed environments (Aare, 2025). Poor normalization and aggregation mechanisms can lead to partial event correlation, events not being detected, sluggish response activities, and in general, poor performance in SIEM. As a result, companies have to implement powerful and scalable normalization methods- usually with machine learning and semantic-aware solutions- to be able to meet the increasing complexity and volume of log data and provide the right and timely security analytics (Chuah et al., 2025; Piet et al., 2025).

2.4 Machine Learning in Cybersecurity and Log Analysis

2.4.1 Overview of Machine Learning Approaches in SIEM

Machine learning (ML) is also a radical technology in cybersecurity, particularly in terms of log data analysis (Sarker, 2023). Organizations can take advantage of the fact that algorithms can learn from past data to automate the detection of the anomalies, predict potential security events, and enhance the overall threat intelligence situation. ML techniques can identify patterns that could indicate some type of malicious action or vulnerability in the system, which would allow organizations to take proactive measures against new threats. The use of ML has been beneficial particularly in the field of cybersecurity due to the massive amount of data generated by various systems. Classical techniques of analysis can hardly keep pace with this amount, and ML is a popular option. Machine learning models have the capacity to examine and handle data in large volumes, and deliver information that would not have been attainable otherwise. As a result, it is possible to not only optimize the performance of security activities using ML systems within the SIEM, but also enhance the efficiency of the incident identification and response systems.

2.4.2 Supervised vs Unsupervised Learning for Logs

The organizations can select supervised or unsupervised learning techniques based on the advantages and disadvantages of each method when using machine learning to analyze the logs. Supervised learning is the learning of a model from a labelled dataset and can be applied for the discovery of known attack and anomaly patterns (Sarker, 2021). The approach is grounded on utilizing known historical data and the model learns through example. Unsupervised Learning, on the other hand, does not involve labeled data and can be used in exploring new patterns and anomalies in the log data that may not have been known before. This is especially helpful in worlds that are dynamic enough that the threats considered are changing at an unacceptable

pace and attack vectors are coming out of the woodwork at pace. The difference between the two approaches and each approach's variations are important to the realization of applicability of each approach to the selection of the best one based on circumstances and nature of the log data being analysed.

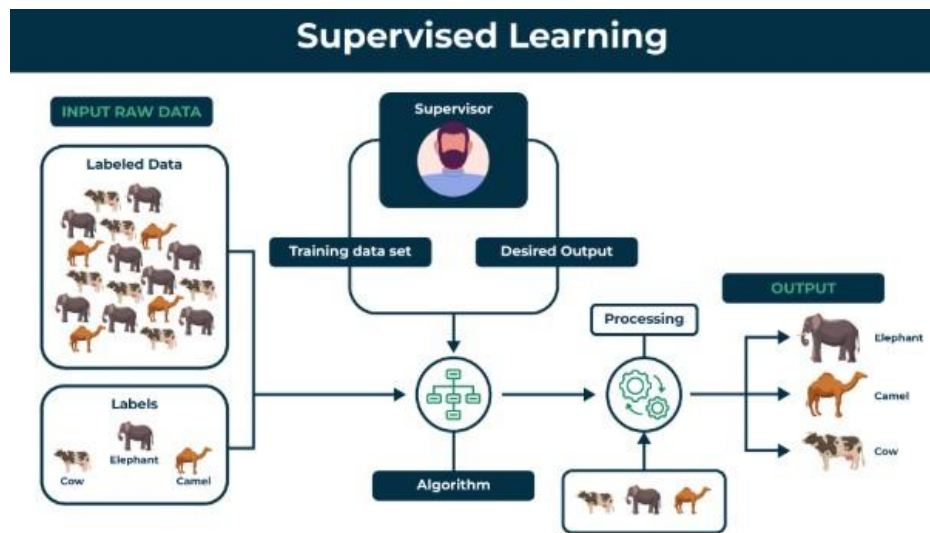


Figure 2.5 Supervised Learning Workflow

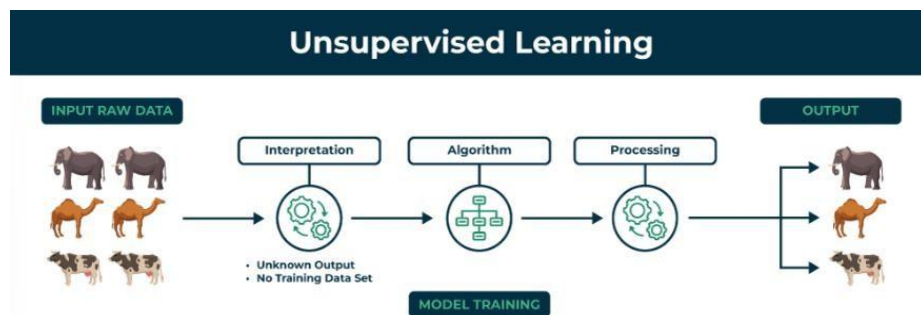


Figure 2.6 Unsupervised Learning Workflow

The workflow of the supervised and unsupervised learning methods of machine learning is shown in figure 2.5 and 2.6 above. In supervised learning, the input information is initially labelled, that is, each data point has known output. The algorithm is trained with these labelled examples, which are used to provide the algorithm with the relationship between the features in the input

and the desired output. When trained, the model can classify or predict the value for new data (data it has never seen before). Conversely, unsupervised learning works using raw data, which does not have any specified classes or output. The algorithm automatically identifies the latent patterns, similarities or the structure from the data, such as clusters or anomalies. Both strategies have their own merit for security/analytics – the supervised learning strategy is useful for identifying the known patterns, the unsupervised learning strategy is useful for identifying the unknown or emergent behaviour.

2.4.3 Challenges of Applying ML to Log Data

Although machine learning could be an effective log data analysis tool, there are a number of challenges that can impede the efficiency of this tool. One of them is data imbalance, in which the amount of normal logs is very large and the amount of abnormal logs is very small (Studiawan et al., 2020). This imbalance can result in imperfect models that will favour the majority class, resulting in low detection for rare but high security event. Moreover, the dimension of the log features may make it difficult to obtain machine learning models since it may cause overfitting and high level of computation. Log data can also be noisy and contain irrelevant data, and more sophisticated feature selection and dimensionality reduction techniques are needed to help the model. Companies should thus consider such challenges keenly by designing their ML pipelines in a manner that the models are strong and capable of identifying the security threats accurately.

2.5 NLP for Log Text Representation

2.5.1 Textual Nature of Log Messages

Usually, log entries are in natural language and it is specially recommended to use Natural Language Processing (NLP) technologies in the

feature extraction (Dwaraki et al., 2020). This textual character gives a chance to use most NLP techniques to interpolate the information of the log entries where the organizations can extract important data out of their logs. Being able to treat the log data as a text allows the security analysts to use linguistic methods to extract keywords, phrases and patterns that could be security incidents. Other characteristics of the log messages may also be known to develop a more sophisticated parsing and analysis methods. To illustrate this, sentiment analysis may be used to identify the tone of entry logs, which may be useful in case of informing about suspicious activity or a potential threat. This approach illustrates how NLP can be applied to improve the interpretability of log data, and the efficiency of the security monitoring process.

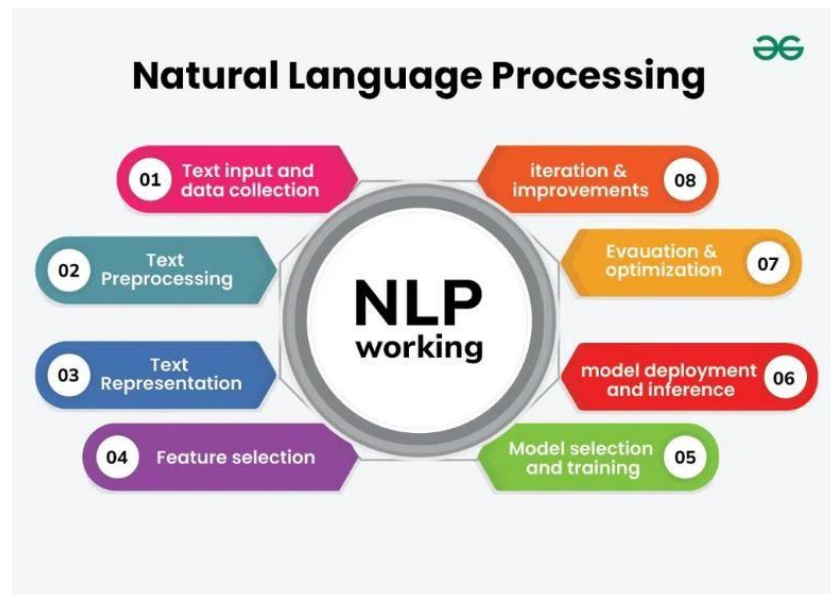


Figure 2.7 Natural Language Processing Workflow

Figure 2.7 demonstrates the end-to-end process of Natural Language Processing (NLP), including the main steps that comprise the process of converting raw textual information into valuable insights. It is initiated by the input of text and data collection and preprocessing of the text, whereby noise in the form of irrelevant symbols and inconsistencies then get eliminated. The

cleaned text is then represented as a text and features are selected to bring out the meaning of the text in a machine-readable format. Subsequently, the models are selected and trained; optimized and then used for inference (classification or pattern detection). Finally, repetition and continual improvement ensures the NLP system improves with the passage of time. It's a systematic development that shows how NLP methodically fills the unorganized written information and smart analytical yields.

2.5.2 NLP Preprocessing Techniques

Log messages are in preparation to be analyzed with NLP preprocessing methods like tokenization and stop word elimination (Parekh et al., 2023). In tokenization, the log entries are divided into a set of tokens or words and then the data can be analyzed in more detail. This is an important step in determining important terms and phrases that are likely to be signs of security incidences. On the other hand, stopword removal consists of eliminating the most basic of words that don't really contribute very much to the analysis, such as the, and, or is. Such stop words could be eliminated and the more meaningful words could be prioritized, thus providing information about the possible security threats. These types of preprocessing methodologies form the basis to further sophisticated NLP operations and extraction of useful features on log data.

2.5.3 TF-IDF and Feature Representation Methods

Term Frequency-Inverse Document Frequency (TF-IDF) and word embeddings are also important feature representation techniques needed to transform log messages into a machine-readable format (Asudani et al., 2023). TF-IDF is a statistical value that is applied to assess the significance of a word within a document in comparison with a set of documents. This way, the terms are highlighted that are more relevant to some entries and help ensure the rest of the analysis is successful. Instead, word embeddings denote words as

continuous vectors space, which reflects their semantic connections. This method can be used to implement this, and aids in models grasping the relationship between different log entries. With such feature representation methodologies, organisations can expect to gain higher accuracy in their log analysis efforts, as well as better performance of their machine learning model.

2.5.4 NLP Challenges in Log Analysis

Although NLP methods have proven useful in log analysis, several issues still exist such as ambiguity, fluctuations in log formats and the need to have domain specific knowledge (Ji et al., 2025). In the logs, technical jargon or abbreviations may be used making the analysis process difficult. Furthermore, the log formats of the different systems could vary, which could make it difficult to use universal methods of NLP, and a specific one might be required to analyse the data successfully. Additionally, NLP models might also perform poorly due to the unavailability of labeled training data, which is generally uncommon in cybersecurity situations. These challenges are of paramount importance in efforts to make the most of NLP techniques in analyzing logs to maximize security surveillance capabilities.

2.6 Ensemble Learning and Random Forest for Classification

2.6.1 Random Forest Fundamentals

Random Forest (Breiman, 2001) builds a large number of decision trees on random subsets of training data and random subsets of features, and combines their predictions by majority voting. The variance and risk of overfitting from a single decision tree is reduced from this design of a bagging, and is comparatively easy to train and tune; moreover, it doesn't require feature scaling or normalisation across heterogeneous feature types, giving it a practical advantage when, as in this project, features range from sparse text-derived features to dense numeric and categorical features (Section 3.4.3).

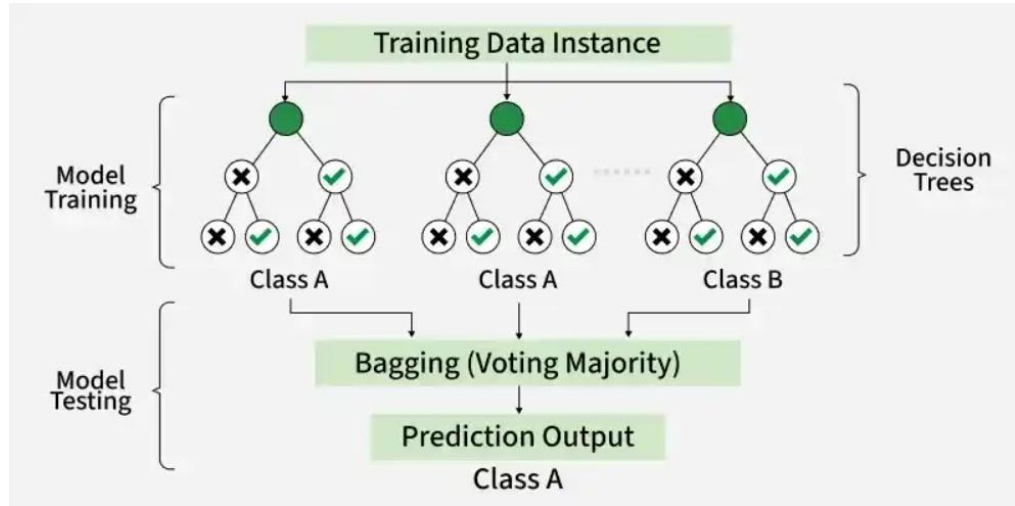


Figure 2.8 Random Forest Classifier

2.6.2 Random Forest in Intrusion and Log Detection

Random Forest has been applied several times to network intrusion and log-based detection problems with very high dimensional heterogeneous feature sets. Abdelaziz et al. (2024) use Random Forest along with permutation feature importance to determine the characteristics of the traffic that most heavily influence malicious activity in a network. Azizi Doost et al. (2025) use Random Forest classification and automated feature extraction to achieve high accuracy and precision in standard intrusion detection benchmarks.

Feature selection and heterogeneous ensemble classification are once again used for Internet of Things (IoT) network traffic in another study by Almotairi et al. (2024) which also achieved enhanced classification accuracy compared to single-model baselines on the IoT network traffic subset - relevant as IoT/DDoS traffic makes up slightly more than half of the unified dataset used in this work (Section 4.2). This body of work shows that Random Forest is competitive in several different security contexts (network intrusion detection, web request detection and IoT traffic detection) and is chosen here as a single classifier that can learn across all three heterogeneous security sources combined in this project, as opposed to having to build a new model for each of them.

2.6.3 Handling Class Imbalance with Ensemble Methods

In addition to raw detection accuracy, ensemble classifiers (ECC) can be paired with class weighting strategies to overcome class imbalance issues as found by Studiawan and Sohel (2020), and results obtained by Almotairi et al. (2024) show that ensemble classifiers combined with feature selection are more effective in detecting the minority attack class in imbalanced network traffic datasets. This ensemble of balanced class weighting and ensemble learning approach has been directly used in this work for the Random Forest configuration (Section 3.4.1) and its impact on the class recall performance of the resulting attack classes is explored empirically in Section 4.5.

2.7 Feature Engineering for Heterogeneous Log Attributes

In real-world log data, only rarely is a single log the only type of data that's included; for instance, security logs might include free text, numeric measurements, and categorical labels in a single log record. The results obtained by Almotairi et al. (2024) indicate that the performance of ensemble classifiers can be enhanced by aggregating feature selection results from different types of attributes for network traffic data. Palma et al. (2025) prove that the performance of a large-scale log analysis can be improved by combining textual and structured representations, rather than only one or the other.

This agrees with the multi-modal feature engineering approach employed in this work where the text features of TF-IDF are encoded with one-hot encoded categorical features and severity and anomaly scores of numerical features in a single feature matrix before classification (see Section 3.4.3). Unlike the case of most other approaches, the relative importance of each feature type is not preconceived, instead measured directly from the feature importance output of the trained model, and the empirical results in Section

4.4.2 confirm that both text and structured attributes are relevant for the classification task, rather than just assuming the relevance of the text features.

2.8 Integration of AI-Based Log Analysis into SIEM

2.8.1 AI-Enhanced SIEM Architectures

The inclusion of AI into the SIEM systems is a powerful addition to the threat detection and incident response capabilities (Ali et al., 2025). The AI-driven systems work on the premise that more sophisticated algorithms are employed for analyzing the log data, giving the organization the ability to spot potential issues in real-time. By integrating into this, normal jobs can be automated and security analysts can focus on more complex investigations. AI enhanced SIEM architectures can feature machine learning models that learn and grow more accurate as additional log data is introduced into the system. This dynamic nature of adaptive approach makes the AI an important element of current cybersecurity practices, as it will make the security systems to be effective despite the changing nature of threats.

2.8.2 Benefits of AI-Based SIEM Systems

The AI-driven SIEM systems offer several benefits, including enhanced accuracy, quicker response times, and improved threat detection (Ndibe, 2025). The analysis of log data can be automated and the patterns and anomalies identified by AI systems are expected to point to the security incidents much faster, enabling organizations to react to the potential threat faster. Moreover, AI-based systems have the potential to give finer details on security events, as a result of which security teams can prioritize their response based on the magnitude and the potential damage of threats. The use of AI in the SIEM solutions would not only enhance the effectiveness of security operations but also contribute to building a more proactive and resilient security strategy.

2.9 Related Works

Table 2.1 Comparison of Related Works

Author & Paper	Problem Addressed	Solution Proposed	Technology Used
Saleem Raja, Vinodini, & Kavitha (2021) – <i>Lexical features based malicious URL detection using machine learning techniques</i>	Detecting malicious requests using only lightweight lexical text features, without deeper content or network-level analysis	Extracts lexical features from raw URL/request text and trains a Random Forest classifier to separate malicious from legitimate requests	Random Forest, supervised learning, lexical text feature extraction
Abdelaziz et al. (2024) – <i>Enhancing network threat detection with random forest-based NIDS and permutation feature importance</i>	Improving network intrusion detection accuracy while retaining interpretability	Trains a Random Forest classifier on network traffic features and uses permutation feature importance to rank the most discriminative attributes	Random Forest, supervised learning, permutation feature importance
Azizi Doost et al. (2025) – <i>A new intrusion detection method using ensemble classification and feature selection</i>	Reducing noise and dimensionality in intrusion detection while maintaining high accuracy	Combines automated feature extraction with a Random Forest classifier to achieve high accuracy and precision on benchmark intrusion datasets	Feature extraction, Random Forest, ensemble classification

Almotairi, Atawneh, Khashan, & Khafajah (2024) – <i>Enhancing intrusion detection in IoT networks using machine learning-based feature selection and ensemble models</i>	Improving intrusion detection performance on heterogeneous IoT network traffic	Applies feature selection together with a heterogeneous ensemble classifier combining multiple base learners	Feature selection, heterogeneous ensemble learning, IoT network traffic
Kalakoti, Vaarandi, Bahsi, & Nõmm (2025) – <i>Evaluating explainable AI for deep learning-based network intrusion detection system alert classification</i>	Improving analyst trust and interpretability of AI-generated intrusion alerts	Evaluates post-hoc explainability techniques for network intrusion detection alert classification and analyst decision support	Explainable AI, alert classification, analyst-facing interpretability

2.9 Summary of Related Works

Overall, these studies are representative of the state-of-the-art of AI-based log processing, namely the unsupervised and representation-learning based processing, as opposed to the rule-based processing that has been the traditional approach. This is highly aligned with the objectives of the project "AI-based methods of aggregating logs used in SIEM systems". The papers reviewed demonstrate that NLP methods (including the use of log parsing, template extraction and clustering strategies can be used to learn latent patterns in heterogeneous and unlabelled log data effectively. With the conversion of raw logs to meaningful embeddings or structured representations, the methods can enable a scalable process for aggregating logs, smart clustering of similar logs, and effective detection of abnormal behaviour without using a predetermined list of rules. From these findings, unsupervised learning with semantic log representations will be adopted in the present project to optimize log aggregation of SIEM, improve the event correlation and aid in automatic anomaly detection and other log-related fields.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter describes the approach in the development of the AI-Based Log Aggregation Techniques for Security Information and Event Management (SIEM). The Agile Methodology is good for maintaining an iterative, flexible development process, which is suitable in the case of an AI-based system that requires continuous improvements. The methodology has six key stages, including requirements, design, development, test, deployment and review. Each stage outlines how logs of different sources can be collected and processed, features can be extracted using NLP (TF-IDF vectorisation), and how structured numeric and categorical features can be added to the logs and used to train a supervised Random Forest classifier to generate an AI-scored, aggregated dataset. Testing, evaluation and iterations of such a system until reaching accuracy, scalability and reliability are also described.

3.2 Project Methodology

3.2.1 Agile Methodology



Figure 3.1 Agile Methodology

As shown in figure 3.1, the Agile development methodology is an iterative development project, so it passes through six phases of iterative development in the project, which are requirements, design, development, testing, deployment and review. Agile also focuses on repetitive cycles unlike in linear development models, which enables the project to go back to previous phases depending on the feedback and results of evaluation. This is the ongoing process which helps in supporting the constant improvement and also makes the system adaptable to changes in requirements, performance and the overall functionality of the system all through the project. Table 3.1 shows the project phase, project tasks and deliverables related to the proposed project.

Table 3.1 Details of Agile Methodology

Phase	Tasks	Deliverables
Requirements	• Identify the problem statement and project objectives • gather requirements for log aggregation, preprocessing, NLP, and supervised machine learning classification • determine scope, limitations, and raw logs to be used • review related research to align techniques and approaches.	• Requirement specification document • defined project scope and objectives • problem statement and justification.
Design	• Design system architecture for log collection, preprocessing, the NLP pipeline, feature engineering, and the Random Forest classification component • define hardware and software requirements • plan data preprocessing steps (cleaning, parsing, normalisation) • develop process flow and system diagrams showing the integration of the Random Forest classification pipeline	• System architecture diagram • hardware and software requirement list • process flow and related diagrams.
Development	• Collect and aggregate raw logs from multiple sources • implement log preprocessing steps (cleaning, tokenisation, parsing) • apply NLP techniques for feature extraction • implement one-hot encoding and numeric feature preparation for structured log attributes; develop and train a Random Forest classifier for log classification (attack vs. normal) and risk scoring; perform iterative improvements	• Functional log preprocessing pipeline • NLP feature extraction module • encoded categorical and numeric feature set • trained and optimised Random Forest model • source code repository

	based on continuous testing feedback.	
Testing	• Test preprocessing accuracy and NLP output quality • evaluate TF-IDF feature quality and combined feature matrix construction • evaluate Random Forest model performance using accuracy, ROC-AUC, precision, recall, and F1-score • compare results with baseline or previous methods; identify issues and refine the system iteratively.	• Test results and analysis • feature engineering validation notes • model performance metrics • improved system after iterations.
Deployment	• Integrate the final aggregated and scored logs into a SIEM-like dashboard environment • conduct final verification using real or simulated log data • finalise project report and presentation materials.	• Deployed aggregated and AI-scored logs from multiple environments • user guideline or manual • final FYP report.
Review	• Evaluate overall log aggregation performance and project objectives achievement • collect feedback and identify limitations of the proposed system • document potential future improvements.	• Project evaluation summary • identified limitations and recommendations • suggestions for future improvements.

3.3 Requirements Phase

The first and most important phase of the project is the requirements phase because it outlines the problem that must be solved and it gives the overall direction of the proposed system. One key problem that was found is

the inability to scale and be effective with the aggregation and analyzing logs in traditional log management systems found in existing SIEM solutions. The volume of logs generated from rapidly expanding IT systems renders manual or rule-based analysis of logs for timely detection of security incidents more difficult.

In this stage, the functional and non-functional requirements were determined and recorded. The functional requirements includes collecting logs from various sources, pre-processing and normalizing the log data, extracting features with the use of Natural Language Processing (NLP), engineering structured numeric and categorical features, and finally, classify log events as attack or normal with the help of a supervised machine learning model. Non-functional requirements are related to the features of the system, such as its scalability, performance efficiency, accuracy, and ability to support various log formats.

A literature review was also conducted to ensure that the approach used was based on the latest research and best practice. As mentioned in Chapter 2, studies on log analysis, NLP-based feature representation, and ensemble learning have supported the technologies employed in this project, and the decision to consider an interpretable supervised classifier instead of other techniques. This also helped to define the scope and limitations of the project including the type of logs to be used (firewall, Android and IoT/DDoS network traffic), system constraints (data set size, computational resources, and project time constraints for feasibility).

Lastly, the expected results of the system were identified, which is the generation of a standardized and unified representation of the log for automated classification and risk score calculation that will facilitate the analysis from a Security point of view. Overall, the Requirements Phase has given the project a solid basis and has ensured that the subsequent phases of the project will be conducted on the basis of clear and realistic requirements.

3.4 Design Phase

Design phase involves converting the requirements in the form of a well-defined and logical architectural design to the proposed solution. At this stage, the overall system architecture was laid out by defining hardware and software requirements, like the programming language, python libraries, development environment, storage requirements, and the prototype for the SOC dashboard for testing and evaluation. These design decisions allowed the system to be technically feasible and in line with project objectives.

At this stage, the logical flow of the system was also determined. The process flow diagrams were developed to illustrate each phase of the pipeline from collecting logs from various sources to performing log preprocessing and schema unification. This design is followed by NLP based feature extraction, which transforms the log messages into structured format, helpful to numerical representation. The derived text-based features are then explicitly addressed and considered, and all the structured numeric and one-hot encoded categorical features were merged into a single feature matrix. Finally, a supervised Random Forest classifier is applied to classify log events as attack or normal, to enable aggregation, correlation, and prioritisation of events based on their expected results and ongoing risk value.

The design allows every component of the system to interact with the others in a coherent manner by establishing the flow of data between components and the responsibilities of each module. This was a step-by-step procedure that made the process of implementation easier, less risk and opportunities for misinterpretation before going into the development stage.

3.4.1 Software Requirements

This project utilizes various programming languages, machine learning libraries, natural language processing tools and development environments for log preprocessing, feature extraction, model training and evaluation. The

software needs of this project and its purpose in the log aggregation system are presented in table 3.2 below.

Table 3.2 Software Requirements

Category	Technology/Library	Function/Features
Programming Language	Python 3	Primary programming language used for log parsing, preprocessing, model development and evaluation.
Machine Learning Library	Scikit-learn	Provides TF-IDF Vectorizer for NLP feature extraction, One-Hot-Encoder for categorical features, Random Forest Classifier for supervised classification, and evaluation metrics (accuracy, classification report, confusion matrix, ROC/AUC).
Numeric Computing	NumPy / SciPy	Supports sparse matrix construction (hstack, csr_matrix) to efficiently combine TF-IDF, numeric, and categorical features.
Data Handling	pandas	Loading, cleaning, transforming, and persisting log datasets (CSV/JSON) across pipeline stages.
Log Parsing Tools	Regex (re module)	Extracts and cleans structured elements from raw log text (leakage-word removal, punctuation stripping, whitespace normalisation).
Serialization	pickle	Persisting the trained Random Forest model, TF-IDF vectorizer, and

		one-hot encoder for reuse without retraining.
Dashboard / UI	Streamlit	Web-based interactive SOC dashboard for filtering, alerting, and reporting on AI-enriched logs.
Development Environment	VS Code	Used for coding, experimentation, and debugging during development.

3.4.2 Hardware Requirements

The hardware employed in this project should be capable of handling processing of logs, NLP preprocessing, and training of machine learning models. Multi-source log data was to be processed by the system, as well as a number of computationally intensive tasks (TF-IDF vectorisation, training random forests) to be performed over a consolidated feature matrix of approximately 94,000 lines. The system needed to be able to process multi-source log data, as well as several computationally intensive tasks, including TF-IDF vectorisation and training random forests over the consolidated feature matrix of approximately 94,000 lines. The system proposed has been designed and simulated for a laptop with the characteristics detailed in table 3.3, with requirements that meet and exceed those presented in the project.

Table 3.3 Hardware Requirements

Component	Specification	Description
CPU	12th Generation Intel(R) Core i7(TM) – 12650H (10 cores, 2.30 GHz)	Provides high-performance multi-core processing suitable for log parsing, NLP preprocessing and machine learning clustering tasks.
RAM	16 GB DDR4	Supports handling large log datasets and ensures

		smooth execution during feature extraction, model training, and clustering without significant system slowdown.
Storage	Minimum 20 GB available (SSD-based system storage)	Stores raw log files, pre-processed datasets, trained machine learning models, and temporary development files.
Operating System	Windows 11 Home Single Language (64-bit)	Provides a stable development environment compatible with Python, machine learning libraries, and virtual environments used in the project.

3.4.3 Flowchart Diagram

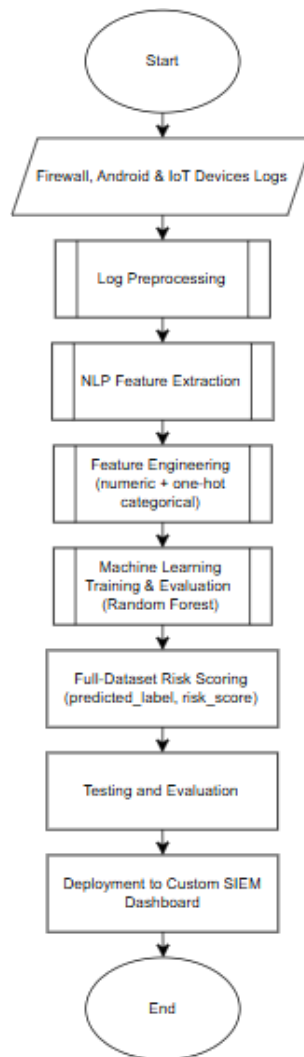


Figure 3.2 Main Flowchart

The overall flow of the implemented AI-based log aggregation pipeline is displayed in Figure 3.2, starting from the ingestion of logs from various sources, to the deployment of the pipeline into the SIEM dashboard. It starts with the collection of three sources of heterogeneous logs: Firewall, Android, and IoT devices. The raw logs are then processed in the Log Preprocessing stage (as shown in Figure 3.3) and are transformed into a common schema and have their labels corrected before they are processed in an exploratory analysis. The preprocessed logs are then fed to the NLP Feature Extraction stage (as shown in figure 3.4), which transforms the content of the free-text logs into numerical vectors using TF-IDF. These textual features are then followed by

structured numeric and categorical features during the Feature Engineering stage as detailed in Figure 3.5, to create one combined feature matrix. This feature matrix is used to train and test a random forest classifier, as described in Figure 3.6, and then the full feature matrix is used to test the trained random forest to generate a predicted label and a continuous risk score for each log event in the full dataset. After the model has been trained, it is then submitted for Testing and Evaluation involving verification of the model's classification performance and the functional behaviour of the overall system, including the SOC dashboard, under a series of testing scenarios, (Chapter 4). Last, the AI-enhanced dataset and trained model artefacts are then placed into the Custom SIEM Dashboard, where security analysts can filter out, monitor, and act on the classified log events.

i) Log Preprocessing

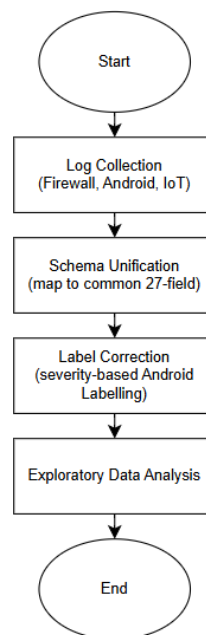


Figure 3.3 Log Preprocessing Flowchart

The Log Preprocessing stage of the main flowchart is broken down into four steps in Figure 3.3. Log Collection starts with three log files, each collected from Android system logs, that are structurally different and each

contain some of the same fields, such as process ID, component, and event template (from an Android system log benchmark); and a third log is a firewall log export containing network-level fields like source IP, protocol, and rule ID; and a fourth log is an IoT/DDoS traffic collection consisting of flow and packet-level statistics, with an attack type label. Almost all of these sources can be mapped to a single schema, 27-field SIEM, due to the next step, Schema Unification, which maps each source to a shared 27-field SIEM schema, meaning that timestamp, source and destination IP, protocol, severity, message, and so on are the same regardless of which system generated the log. The Label Correction step is added since there is no reliable ground-truth attack label available in the Android source after unification, it derives an attack label based on severity; the latter are labelled as attack, whereas others are labelled as normal. Finally, an Exploratory Data Analysis step is conducted on the gathered and relabelled dataset to provide descriptive statistics and bar-chart visualizations for distributions of source, label, severity, protocol and attack-type, to ensure that the dataset is well-formed and can be used for feature extraction and model training.

ii) NLP Feature Extraction

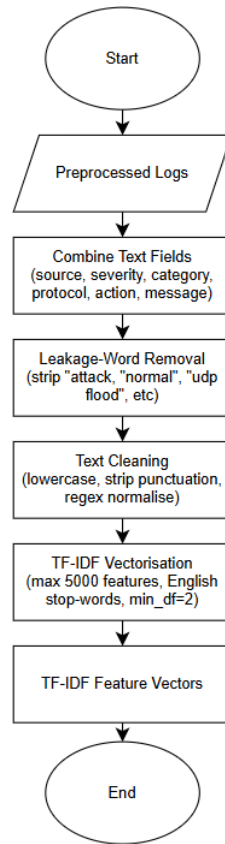


Figure 3.4 NLP Feature Extraction Flowchart

Figure 3.4 illustrates the process of converting the preprocessed log data shown in Figure 3.3 into numerical features that are appropriate for machine learning purposes. For each log events, a string of text is generated from six descriptive text fields: source type, severity, event category, protocol, action, message. The literal phrases in the text (e.g., “attack”, “normal”, “udp flood”, “syn flood”, and “no attack”) are explicitly removed from the text before this text is vectorised. This is because some of the original log messages contain the ground-truth verdict in the log message itself, and if the log message is untouched, the subsequent model will learn to read the answer in the log message, rather than learning really predictive patterns, and end up with an artificially high and unrealistic level of accuracy. The text is then cleaned, by converting it to lowercase, removing non-alphanumeric characters and punctuation with regular expressions and normalizing the whitespace. Finally,

TF-IDF Vectorisation is performed with a maximum of 5,000 features, English stopword removal and minimum document frequency of 2, to convert the text into TF-IDF Feature Vectors to feed as an input to the machine learning model.

iii) Feature Engineering

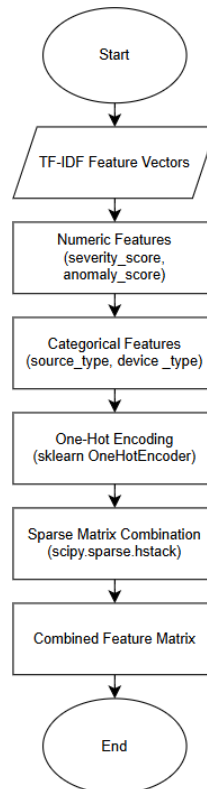


Figure 3.5 Feature Engineering Flowchart

Figure 3.5 illustrates the text feature generation results from Figure 3.4 and the structured log attributes that are used to compose the final feature matrix for model training. Two numeric fields (`severity_score` and `anomaly_score`) are cleaned and used as Numeric Features. The three categorical fields (`source_type`, `protocol`, and `device_type`), are treated as Categorical Features, requiring a One-Hot Encoding; one-hot encoding is a technique that turns each category into a binary indicator field, and does not raise an error if the unseen value of the categorical feature is encountered at inference time. These three feature blocks are then combined using Sparse

Matrix Combination which simply concatenates the three feature blocks into one matrix without having to convert the huge sized sparse TF-IDF feature block into a dense one, therefore saving memory usage across the entire 94,162 rows of data. The Combined Feature Matrix will be the output of this stage, where each line will contain all the text-derived, numeric, and categorical features of each log event, ready for classification.

iv) Random Forest Classification

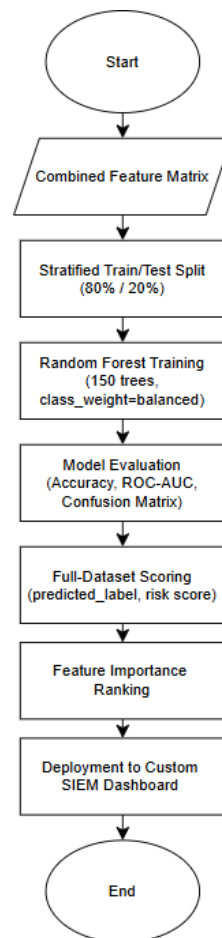


Figure 3.6 Random Forest Classification Flowchart

The supervised classification step in the pipeline is outlined in Figure 3.6. The Combined Feature Matrix produced as in Figure 3.5 is split into two sets, 80% for training and 20% for testing in such a way that the proportion of

attack and normal events is maintained in both sets. An ensemble of 150 decision trees is then built to the training data in a Random Forest Training step with balanced class weighting to minimize class bias towards the majority (normal) class. When the model is trained, it is tested on the held-out test split during Model Evaluation, and the accuracy, ROC-AUC and a confusion matrix is computed to see how well the model generalises to unseen data. After scoring the model, it is applied to the whole dataset in the Full-Dataset Scoring step, and the predicted_label and the continuous risk_score are given to all the log events, including the training log events. To meet the explainability needs mentioned in Section 2.8, a Feature Importance Ranking is directly produced from the trained Random Forest, indicating the relative importance of the text, numeric and categorical features in ranking the feature importance. Lastly, the trained model, its evaluation and the fully scored dataset are passed to the Custom SIEM Dashboard for Deployment, where the results are shared with security analysts.

3.4.4 Entity Relation Diagram (ERD)

The implemented system stores data as a sequence of versioned, flat CSV/JSON files rather than related database tables, so its data design is best expressed as a schema and pipeline diagram rather than a relational entity-relationship diagram. Figure 3.7 shows how the three raw log sources are mapped onto one unified 27-field SIEM schema, corrected for Android labelling, and then enriched by the TF-IDF and Random Forest pipeline into a 28-field AI-enriched output consumed by the SOC dashboard.

Figure 3.7 - Data Schema and Pipeline Diagram (AI-Based Log Aggregation for SIEM)

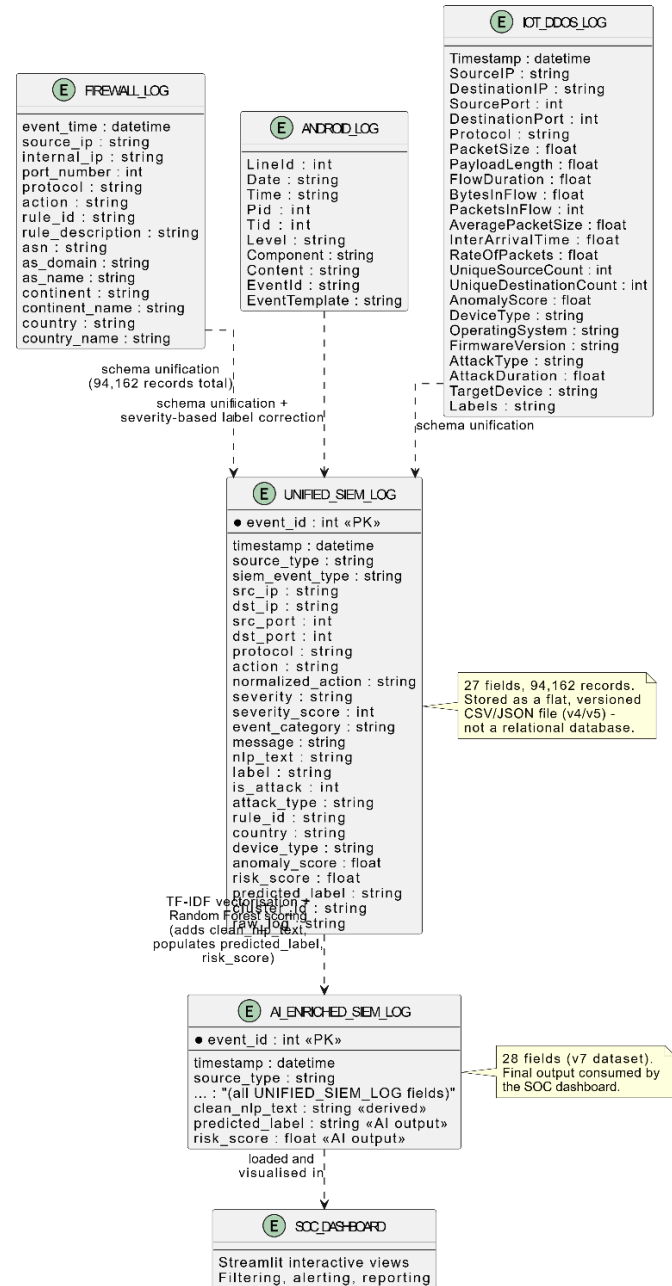


Figure 3.7 Entity Relationship Diagram (ERD)

Table 3.4 Entity Relationship Diagram Details

Field	Type	Role
event_id	int	Unique identifier for each log event, generated during schema unification.
timestamp	datetime	Event time, parsed and used for time-window filtering in the dashboard.
source_type	string	Origin of the record: firewall, android, or iot.
src_ip / dst_ip	string	Source and destination IP addresses, used for network-level correlation.
protocol	string	Network protocol associated with the event (e.g., TCP, UDP, ICMP).
severity / severity_score	string / numeric	Categorical and numeric severity of the event.
message / nlp_text / clean_nlp_text	text	Raw and cleaned free-text content used for TF-IDF vectorisation.
label / is_attack	string / int	Ground-truth attack/normal label used to train the classifier.
attack_type	string	Specific attack category, where available (e.g., SYN Flood, UDP Flood).
anomaly_score	numeric	Unsupervised anomaly signal carried over from the IoT source data.
predicted_label	string	Random Forest's predicted attack/normal classification.
risk_score	numeric	Continuous attack-class probability output by the Random Forest model.

3.5 Development Phase

The development phase is a phase focused on implementing the designed system components, and adding the components to the functioning pipeline. This step starts with the gathering and consolidation of unstructured logs of a number of pre-specified sources including firewall, Android and IoT devices. These records are stored in a centralized storage for further processing.

Then, the log preprocessing methods are used to prepare and standardize the raw data. This includes removing duplicate data, decoding log entries, tokenizing the text data and translating the different log records into a standard format. NLP methods are used to extract meaningful features in the log messages after preprocessing is done. The methods convert text logs to numeric vectors which can contain contextual and semantic information.

Once NLP features are extracted, numerical fields are cleaned and one-hot encoding is performed on categorical fields like source type, protocol and device type. The TF-IDF text features are added along with these structured features in a sparse feature matrix. This is then passed to a Random Forest classification algorithm, which is then trained to classify attack events as such, using an 80/20 split, with the 20% of the data being kept as a test set. At this phase, development is repeated: hyperparameters of the model (parameters of classifiers, image features, etc.) are iteratively optimized based on the results of the evaluations (accuracy, ROC-AUC, feature importance, etc.). Persisting the trained model, TF-IDF vectorizer and one-hot encoder as serialized objects, along with the versioned dataset outputs at each pipeline stage, ensures reproducibility and maintainability.

3.6 Testing Phase

Testing Phase is done to assess the accuracy, efficiency and functionality of the implemented system. The testing involves passing the validation tests to make sure that the preprocessing pipeline is correct enough that the logs are indeed being cleaned, parsed and normalized correctly. The quality of the NLP features extraction is then determined to establish if meaningful information has been maintained.

The quality of the combined feature matrix is also checked, such as the shape and sparsity of the TF-IDF, numeric, and one-hot encoded parts, and that literal label-revealing words were removed from the text before vectorisation, to prevent "label leakage". The accuracy, ROC-AUC, precision, recall and F1-score are used to evaluate the performance of the Random Forest classifier on

the held-out test split, a confusion matrix and ROC curve are also provided. Results are compared to the model's own feature importance ranking to ensure that both text-derived and structured features are important for classification, and if problems surface, then it is followed up with an iterative process of refining the pre-processing steps or model parameters.

Then the machine learning clustering models are tested using the relevant performance criteria such as accuracy, precision, recall, F1 score etc. After that the machine learning clustering models are tested by using the relevant performance criteria like accuracy, precision, recall, F1 score... etc. The performance of the results obtained from clustering is compared with the baseline procedures or the conventional log methods of analysis to indicate the performance improvement. If any issues are identified, it is then followed by an improvement of the potential pre-processing techniques, model parameters or system configurations.

3.7 Deployment Phase

The deployment phase involves integrating the entire system into a SIEM-like system or prototype. In this phase, all stages of the pipeline (log collection, preprocessing, NLP processing, and ML clustering) are coded and evaluated with real or simulated logs. The final verification is performed to verify that the system works as expected in the real conditions. User documentation and usage guidelines are prepared to explain how the system works, how to set up the system and how to interpret the results. This step also involves the completion of the project report and making up of presentation materials to sum up the results of system design, implementation and evaluation.

3.8 Review Phase

The last step of the methodology is the review phase which is aimed at assessing the effectiveness of the proposed system as a whole and reflections on the outcomes of the projects. During this stage, the performance of the system is compared with the targets and needs that are set during the Requirements Phase. This testing is done to find out whether the system is effective in enhancing the efficiency of log aggregation, scalability and the automated analysis over the conventional SIEM strategies.

The feedback is received in the form of the results of system observation and testing to single out the advantages and drawbacks of the proposed approach. The constraint of datasets, computational costs, model generalization and challenge of deployment on a real-time basis are critically evaluated. These restrictions have been recorded in order to offer transparency and academic rigor.

At the end, some lessons that have been learned during the project are summarized and possible future improvements suggested. They can be the extension of the system to analyze other log sources, a real-time streaming analysis, or exploring deeper learning models. The Review Phase makes sure that the project will have significant contributions and introduce a definite direction of future research and development.

3.9 Project Milestones

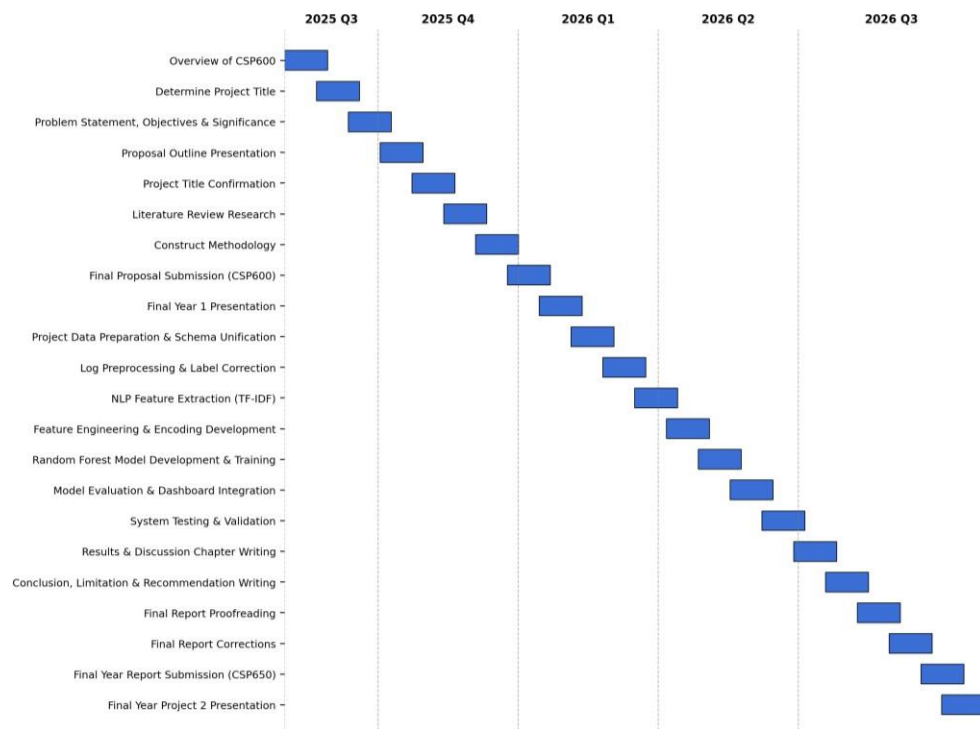


Figure 3.8 Gantt Chart

The project schedule in the form of a Gantt chart was created using the Agile approach and is presented in Figure 3.8. The overall chart detailing the chronological and time-based step-by-step exploration of the project activities from the initial planning process through to the Final Year Project 2 presentation, may suggest more of an iterative and incremental development approach rather than a strict linear approach.

The activities of this project are mapped out in the Agile process stages which are Requirements Analysis, System Design, Development, Testing, Evaluation and Review. The requirements related milestones that are followed in the pipeline (data preparation, NLP feature extraction) and the model development (Feature Engineering & Encoding Development, Random Forest Model Development & Training, Model Evaluation & Dashboard Integration) milestones are iteratively developed and tested throughout the process. Like in the project's lifecycle, the Gantt chart provides a convenient timeline of tasks and their durations, and yet is both agile and updated.

3.10 Summary

The project described uses the AI method to improve the log aggregation from to Security Information and Event Management (SIEM) process. Log data comes from multiple and diverse sources such as Android, IoT devices and firewall. The first steps in the processing of the raw logs are data cleansing, parsing and normalization. An NLP engine is then used to identify meaningful information in unstructured log messages enabling significant patterns and other contextual information to be identified.

Then the Remaining TF-IDF text features and structured numeric and one-hot encoded categorical features are merged together into a combined feature matrix. This feature matrix is then given to a supervised Random Forest classifier to identify attack events and to assign a continuous risk score to each log entry without using unsupervised clustering to determine membership in a group. The approach proposed has been assessed by comparing its performance in the unification of different log formats and in the log analysis using SIEM, with the traditional aggregation methods.

CHAPTER 4

RESULT, DISCUSSION AND FINDINGS

4.1 Introduction

In this chapter, the dataset generated by the AI-based log aggregation pipeline is introduced, the test cases used to validate the system are described and the quantitative results from the trained Random Forest classifier are reported and analyzed. The chapter is divided into four parts: the constitution of the final unified dataset (Section 4.2), the functional and model testing scenarios run on the system (Section 4.3), the held-out evaluation and full-dataset scoring results (Section 4.4), and a discussion of the implications of the results for the strengths and limitations of the current implementation (Section 4.5).

4.2 Data Description

These three raw log sources were combined during the design and development processes outlined in Chapter 3, namely firewall logs (around 44,000 rows of network level event data, including source IP, protocol, action, and rule ID), Android logs (around 2,000 rows of endpoint system level event data), and IoT Devices logs (around 48,000 rows of flow and packet level data statistics with an attack type label). These sources were mapped to a common schema of 27 fields with SIEM, and then merged together into a single unified set of 94,162 events with the severity-based label correction for the Android source. The composition of this dataset by source, ground-truth label and severity is summarised in Table 4.1, Table 4.2, and Table 4.3 respectively.

Source Type	Event Count	% of Total
IoT (INDDOS24)	48,192	51.2%
Firewall	43,970	46.7%
Android	2,000	2.1%

Table 4.1. Event count by log source in the unified dataset.

Ground-Truth Label	Event Count	% of Total
Normal	57,580	61.1%
Attack	36,582	38.9%

Table 4.2. Ground-truth label distribution across the unified dataset.

Severity	Event Count	% of Total
Low	55,909	59.4%
Medium	19,445	20.7%
High	15,578	16.5%
Critical	3,230	3.4%

Table 4.3. Severity distribution across the unified dataset.

This composition leads to two observations. First, the IoT and firewall sources make up more than 97% of all events, and Android makes up only 2.1%, which is an unbalanced representation of type of sources, revisited in Section 4.5 with regard to feature importance. Second, the statistics for the 59.4% of events classified as low severity and 38.9% of events classified as attack overall align with the SIEM literature discussed in Section 2.2.2: that severity and the ground truth (whether it's attack or normal) are related but distinct signals. The almost 3:2 ratio of normal to attack events also validates the per-expected class imbalance that was found in Section 2.4.2 and which will be addressed by the class-weighted training strategy used in Section 3.4.1.

In terms of the individual predictions of the trained models, these three sources show very different behavior, as described in the following in Section 4.2.1 to 4.2.3 and summarised in Table 4.4.

Source	Events	Predicted Attack Rate	Mean Risk Score	Median Risk Score
Firewall	43,970	10.01%	0.100	0.000
Android	2,000	0.15%	0.002	0.000
IoT	48,192	67.53%	0.716	0.860

Table 4.4. Per-source prediction and risk score summary

The difference in the risks between sources is impressive with a mean risk score from 0.002 for Android to 0.716 for IoT, representing a difference of over 350-fold. This pattern is explored in detail in the next subsections and is investigated as direct evidence for the `anomaly_score` dominance found in the feature importance analysis (Section 4.4.2) in Section 4.5.

4.2.1 Firewall Log Source

The firewall source is from an export of the firewall logs in 2022 and contains 43,970 events, representing 46.7% of the unified dataset. All of its native fields are network based: `event_time`, `source_ip`, `internal_ip`, `port_number`, `protocol`, `action`, `rule_id`, `rule_description`, and `geolocation/ASN` metadata like `country`, `continent`, `autonomous_system_name`, which all lack the native fields of the Android or IoT sources, that is why schema unification was necessary in Section 3.4.4.

Table 4.5 shows the percentage of the subset of the firewall that was mapped into the unified schema and that was assigned a score by the trained Random Forest model. In this case the severity is binary (low or high) and there is no event occurring in the middle of the range of severity; therefore, the number of events of high severity (4,401) corresponds almost exactly to the number of events that have been predicted as attack (4,401) without a medium or critical event. The mean risk score of 0.100 indicates that the majority of firewall traffic is in a grey area, means the model classifies the vast majority of traffic as low-risk (mean predicted attack rate of only 10.01%).

Metric	Value
Total events	43,970 (46.7% of dataset)
Severity: low / high	39,569 (90.0%) / 4,401 (10.0%)
Predicted attack rate	10.01% (4,401 events)
Mean / median risk score	0.100 / 0.000
Most common action	allow (26,523 events, 60.3%)
Least common top-6 action	warning (4,262 events, 9.7%)

Table 4.5. Firewall log source summary statistics.

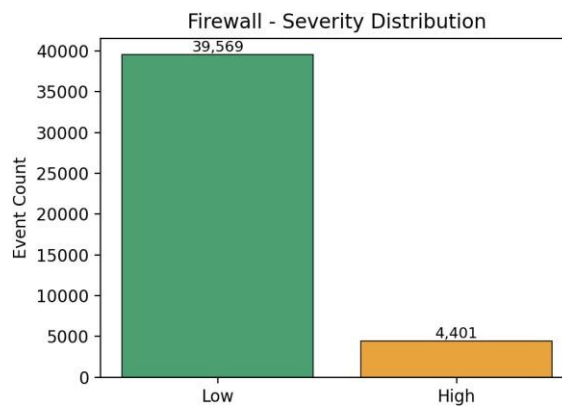


Figure 4.1. Firewall severity distribution.

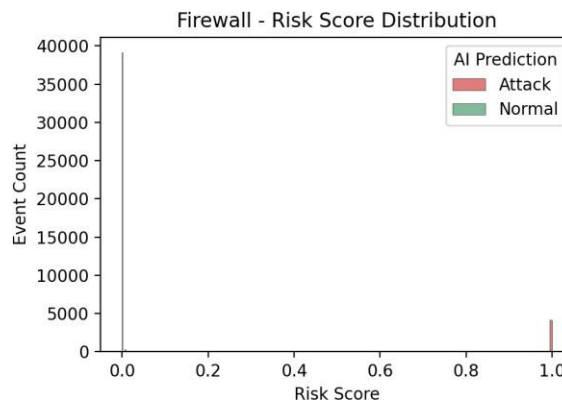


Figure 4.2. Firewall risk score distribution by AI prediction.

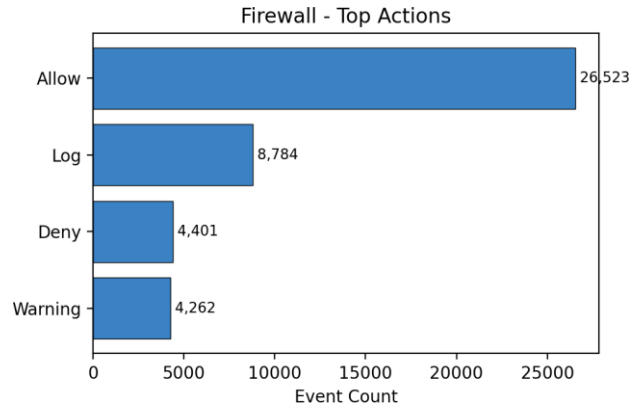


Figure 4.3. Firewall top actions

Figure 4.3 shows that allow (60.3%) actions make up the bulk of firewall traffic, followed by log (20.0%) actions, and almost exactly the same proportion of almost exactly the same severity of events occur in the deny (10.0%) category, as in the high-severity and predicted-attack categories in Table 4.5. The alignment of the deny action, high severity and positive attack prediction indicates that the model has captured a meaningful and internalised notion of what is considered “firewall-side malicious activity” rather than a distribution of risk scores that do not have any relationship with the semantics of the logs.

4.2.2 Android Log Source

The basic source for the Android is from the Android_2k structured log benchmark and is by far the smallest of the 3, with only 2,000 events (2.1% of the unified source). These native fields are sourced from Android's system logging, and describe operating system level activity, rather than network traffic, and as mentioned in Section 3.5, did not have any useful attack label which can be extracted from the data until the severity-based relabelling performed in this project.

As illustrated in Table 4.6, the overwhelming majority of events on Android are benign, with 91.4% being low severity and only 3 high or critical events (0.15%) predicted as attack. Only the 3 events with high severity,

relabel_android() (Section 3.5) would be given an attack label, which is exactly the number of high severity events seen in Table 4.6. The mean risk score of 0.002 is close to zero, which is due to the low frequency of high severity Android events and the resulting low frequency of high anomaly_score events for the source type, as risk_score is directly derived from the anomaly_score signal available for IoT traffic, which is not the case for Android traffic.

Metric	Value
Total events	2,000 (2.1% of dataset)
Severity: low / medium / high	1,827 (91.4%) / 170 (8.5%) / 3 (0.15%)
Predicted attack rate	0.15% (3 events)
Mean / median risk score	0.002 / 0.000
Most common event category	phonestatusbar (507 events, 25.4%)
Second most common	powermanagerservice (387 events, 19.4%)

Table 4.6. Android log source summary statistics

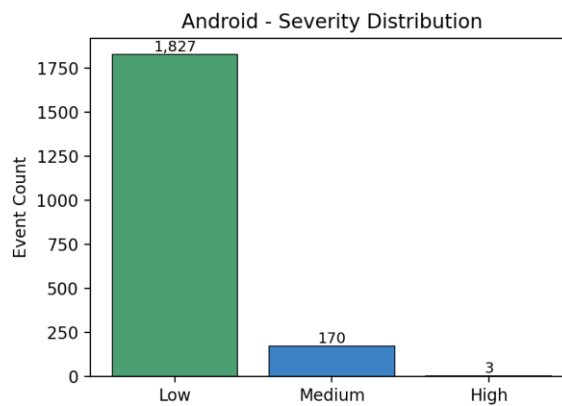


Figure 4.4. Android severity distribution

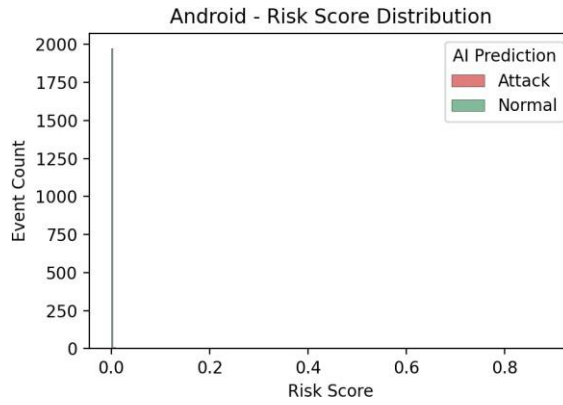


Figure 4.5. Android risk score distribution by AI prediction.

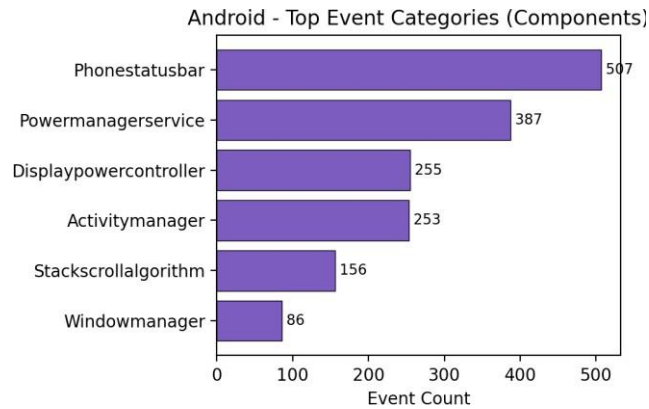


Figure 4.6. Android top event categories (components).

The most common categories for Android events are listed in Figure 4.6: `phonestatusbar`, `powermanagerservice`, `displaypowercontroller`, `activitymanager`, `stackscrollalgorithm`, `windowmanager`. They are all UI and power-management components, not security-related, so it is not surprising that there are few Android events of high severity under the relabelling rule. The near zero risk score for this source (Table 4.6) is therefore not considered a model weakness, but a true reflection of the underlying benchmark data, which was not originally designed to be a security incident database (as discussed further in Section 5.4).

4.2.3 IoT Log Source

The IoT source comes from the INDDOS24 data source and is the largest of the three, accounting for 48,192 events (51.2% of the joined data source). Its native fields are network flow and packet level statistics, including source and destination IP and port, protocol, packet size, payload length, flow duration, bytes and packets in flow, inter-arrival time, unique source and destination counts, device type, operating system, firmware version, and, unlike the other two sources, a pre-existing Anomaly Score and Attack Type label.

As can be seen in Table 4.7, the character of IoT traffic is very different from the other two sources, with severity being distributed across all four levels (as compared to clustered at the low end with the firewall source), and 67.53% of events predicted as attack, resulting in a mean risk score of 0.716 – significantly higher than that of the firewall source, and over 350 times higher than the Android source (Table 4.4). This is the direct empirical result of the `anomaly_score` feature being identified as the most important predictor in Section 4.4.2: this feature is unique to the IoT source, and its presence is what makes the IoT risk scores much higher than that of the other two sources, which do not contain a corresponding signal.

Metric	Value
Total events	48,192 (51.2% of dataset)
Severity: low / medium / high / critical	14,513 (30.1%) / 19,275 (40.0%) / 11,174 (23.2%) / 3,230 (6.7%)
Predicted attack rate	67.53% (32,545 events)
Mean / median risk score	0.716 / 0.860
Top attack types (predicted attacks only)	SYN Flood (15,208), UDP Flood (15,089)
Most common protocol	ICMP (16,153 events, 33.5%)

Table 4.7. IoT log source summary statistics.

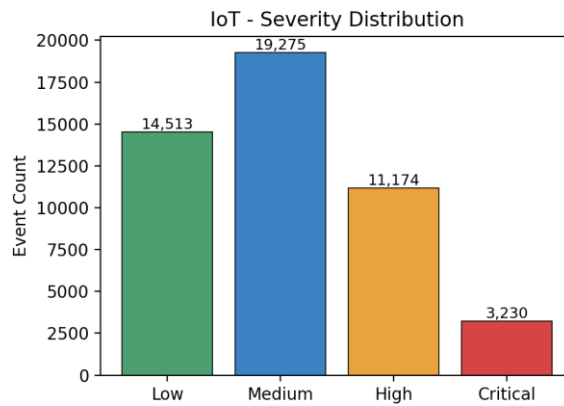


Figure 4.7. IoT severity distribution.

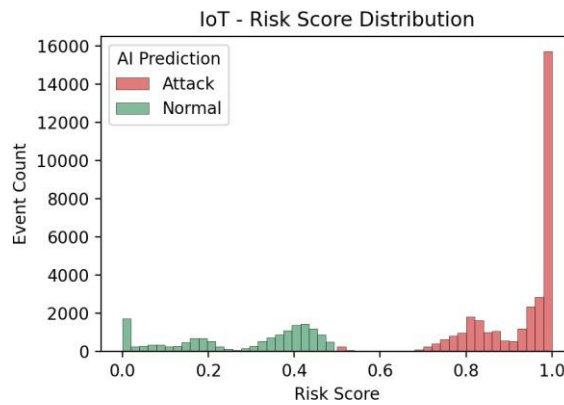


Figure 4.8. IoT risk score distribution by AI prediction.

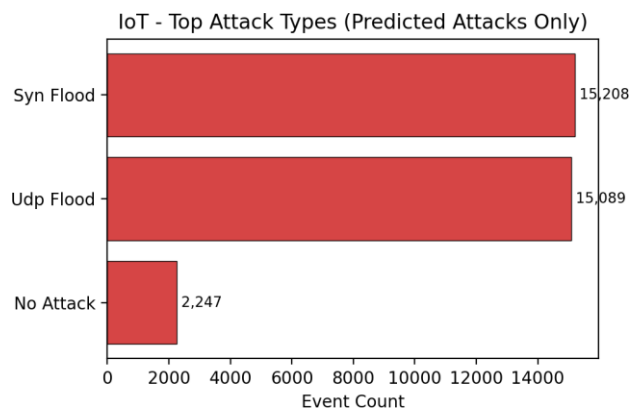


Figure 4.9. IoT top attack types among predicted attacks.

As evident from Figure 4.9, some of the 32,545 IoT events predicted as attack include an original attack_type of “no attack” in the source INDDOS24 dataset (6.9%). That is, the trained Random Forest model disagrees with the attack-type annotations provided by the raw dataset itself on a small but non-

trivial number of events, which are classified as a malicious attack by the trained model but not by the raw dataset. This may be either a real false positive caused by the `anomaly_score` being high due to unusual but legitimate traffic or a false positive because the model has identified a true anomaly that wasn't represented in the dataset by an `attack_type`. This mismatch can only be discovered by looking at the IoT source individually, and not just as an aggregate in this way, and is pointed out as a limitation in Section 5.4.

4.3 Testing Scenarios

As in the Testing Phase (Section 3.6), the system was evaluated at the SOC dashboard prototype where it is deployed as the SIEM (Section 3.7) and model level testing of the trained classifier was done on a held-out data split. The test scenarios conducted, steps taken, the expected result and the observed result is shown in table 4.8. The following scenarios (T1-T6) demonstrate the SOC dashboard's functionality, which is ultimately used by an analyst and consumes the AI-enhanced dataset.

Table 4.8. Model testing scenarios executed against the implemented system.

ID	Scenario	Test Steps	Expected Result	Status
T1	Dashboard authentication	Attempt sign-in with valid and with invalid credentials	Valid credentials load the dashboard; invalid credentials are rejected with an inline error	Pass
T2	Combined dashboard filtering	Apply source, severity, and risk-score range filters together	Only records matching all active filters are shown; KPI cards recompute accordingly	Pass

T3	Empty filter result handling	Apply a filter combination with no matching records	An explicit empty-state message is shown instead of an unhandled error	Pass
T4	Alert threshold sensitivity	Lower the high-risk alert threshold slider from 0.80 toward 0.30	The active alert count increases monotonically as the threshold decreases	Pass
T5	Source-specific monitor tabs	Open the Android, Firewall, and IoT monitor tabs	Each tab shows KPIs and charts limited to its own source subset	Pass
T6	Filtered data export	Apply filters, then download the filtered CSV/JSON	Downloaded file row count matches the currently filtered record count	Pass

4.4 Results

4.4.1 Model Evaluation Results (Held-Out Test Set)

The Random Forest classifier was tested on a 20% held-out test set of 18,833 events that were selected with stratification. The accuracy, ROC-AUC and per-class precision, recall and F1-scores are reported in Table 4.9.

Table 4.9. Random Forest evaluation results on the 20% held-out test split.

Metric	Value	Notes
Accuracy	79.81%	Overall held-out test-set accuracy (18,833 events)
ROC-AUC	89.76%	Area under the ROC curve for the attack class
Attack precision / recall / F1	73.88% / 74.29% / 74.09%	Class 'attack', support = 7,317

Normal precision / recall / F1	83.61% / 83.31% / 83.46%	Class 'normal', support = 11,516
Macro-average F1	78.77%	Unweighted average across both classes

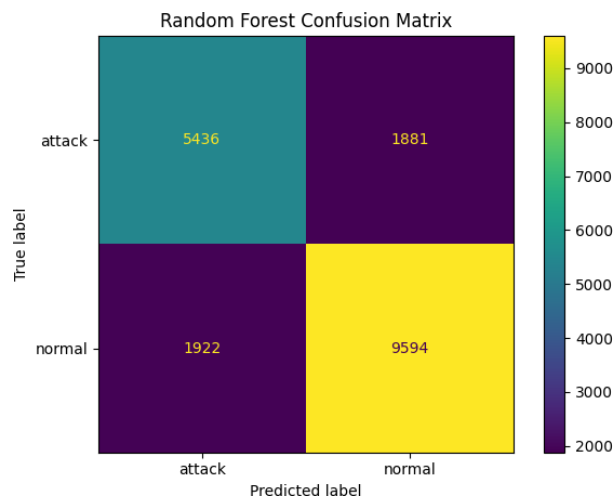


Figure 4.10. Random Forest confusion matrix on the held-out test set.

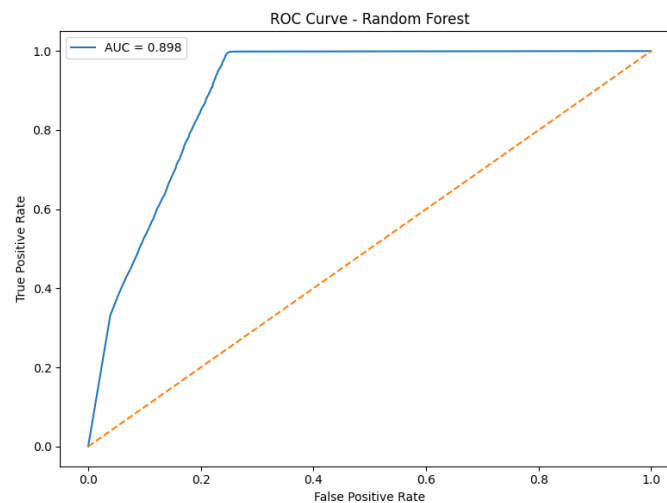


Figure 4.11. ROC curve for the Random Forest classifier (attack class).

These results show that the model is consistently better for the majority (normal) class compared to the minority (attack) class for all the metrics, a difference that will be explored in greater detail in Section 4.5.

4.4.2 Feature Importance

Table 4.10 presents the top 10 most important features with respect to the Random Forest feature importance. The `anomaly_score` field, which is an unsupervised signal passed from the IoT source data, is the dominating field with around 45% of the total relative importance, much higher than any other individual field.

Rank	Feature	Relative Importance
1	<code>anomaly_score</code>	0.451
2	<code>firewall (source_type text token)</code>	0.048
3	<code>severity_score</code>	0.042
4	<code>low (severity text token)</code>	0.039
5	<code>iot (source_type text token)</code>	0.034
6	<code>device_type_firewall</code>	0.033
7	<code>source_type_iot</code>	0.032
8	<code>traffic (text token)</code>	0.031
9	<code>block (text token)</code>	0.026
10	<code>network (text token)</code>	0.025

Table 4.10 Top 10 Random Forest feature importances

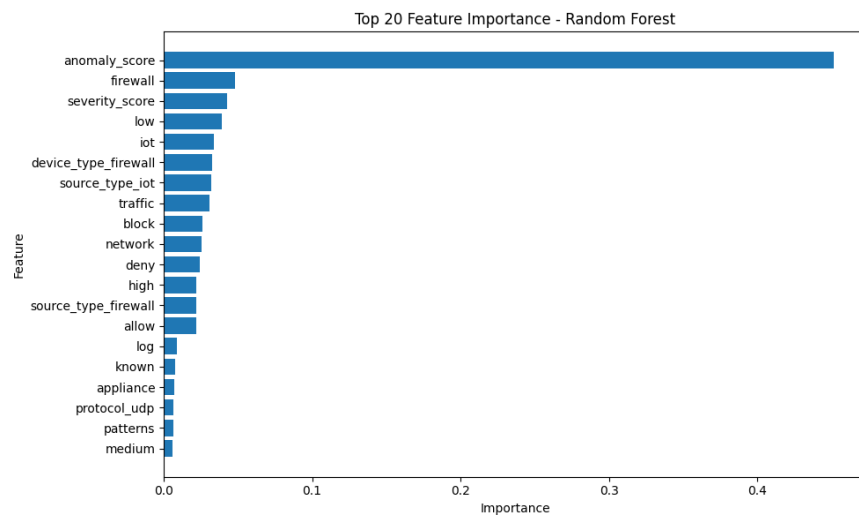


Figure 4.12 Top 20 Random Forest feature importances.

Both the structured attributes (severity_score, device_type_firewall, source_type_iot) and the text-derived TF-IDF tokens (firewall, low, iot, traffic, block, network) are present in the top 10, which is a direct confirmation of the multi-modal feature engineering hypothesis mentioned in section 2.7: none of the text or structured attributes alone would have been enough to result in this top 10.

4.4.3 Full-Dataset Scoring Results

The trained model was re-applied to the entire 94,162 row data set – including the rows used for training the model – to fill in predicted_label and risk_score for each event. Table 4.11 shows the resulting prediction distribution.

Metric	Value
Predicted 'normal' (full dataset)	57,214 (60.8%)
Predicted 'attack' (full dataset)	36,948 (39.2%)
Mean risk_score (full dataset)	0.413
Median risk_score (full dataset)	0.200
label vs. predicted_label agreement (full dataset)	95.62%
Accuracy on held-out test set only (Table 4.9)	79.81%

Table 4.11 Full-dataset prediction summary versus held-out test accuracy

4.4.4 SIEM Dashboard

The AI-enhanced dataset created in Section 4.4.3 is not meant to be directly processed by a security analyst as a CSV file, but it is deployed in an interactive Security Operations Centre (SOC) dashboard that was developed using Streamlit, as described in Section 3.7. The final presentation layer of the pipeline is the Dashboard, and it's the only one where all the other results in this chapter (classification output, risk scores, feature importance, and per-source behaviour) can actually be made available to an end user. The access is secured with a login screen and after successfully logging in the analyst is

greeted with a live KPIs summary bar and 9 analytical tabs as listed in Table 4.12.

Tab	Purpose and Key Elements
Overview	High-level summary: total logs, unique source/destination IPs, active alerts, an event-volume timeline, an average-risk gauge, and attack-vs-normal, severity, and source-distribution charts for the currently filtered data.
Threat Analysis	A live confusion matrix and precision/recall figures recomputed on the current filter (distinct from the fixed held-out metrics in Table 4.9); an Explainable AI panel that narrates, in plain language, why the single highest-risk event was flagged; the Random Forest feature importance chart (Figure 4.12); and a supervised-vs-unsupervised risk comparison scatter plot.
Defense	A simulated SOC alarm trigger, a live active-alert queue with severity colour-coding and suggested response actions per event, and a simulated or live SOC email alert workflow.
Incident Response	A ticketing workspace that converts high-risk alerts into investigation tickets, each with a generated ticket ID, an approximate MITRE ATT&CK technique/tactic mapping, and analyst-editable status, priority, and notes fields.
Firewall Monitor	Source-specific KPIs, severity distribution, AI prediction split, top attack types, and risk score distribution restricted to firewall events only, matching the statistics reported in Section 4.2.1.
Android Monitor	The same source-specific view as above, restricted to Android events only, matching Section 4.2.2.

IoT Monitor	The same source-specific view as above, restricted to IoT/DDoS events only, matching Section 4.2.3.
SOC Report	An auto-generated executive summary (security posture rating, threat snapshot, model performance summary, recommended SOC focus areas) that can be downloaded as a plain-text report.
Explorer	A searchable, filterable table of raw AI-enriched records with CSV/JSON export and per-column summary statistics.

Table 4.12 SIEM dashboard tabs and their key features

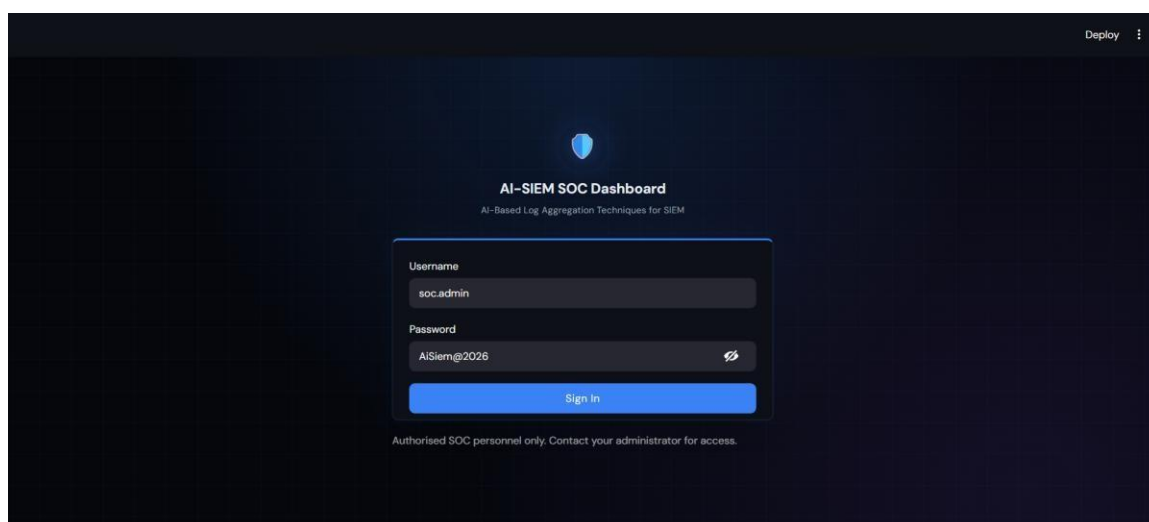


Figure 4.13 SIEM Dashboard Login Page

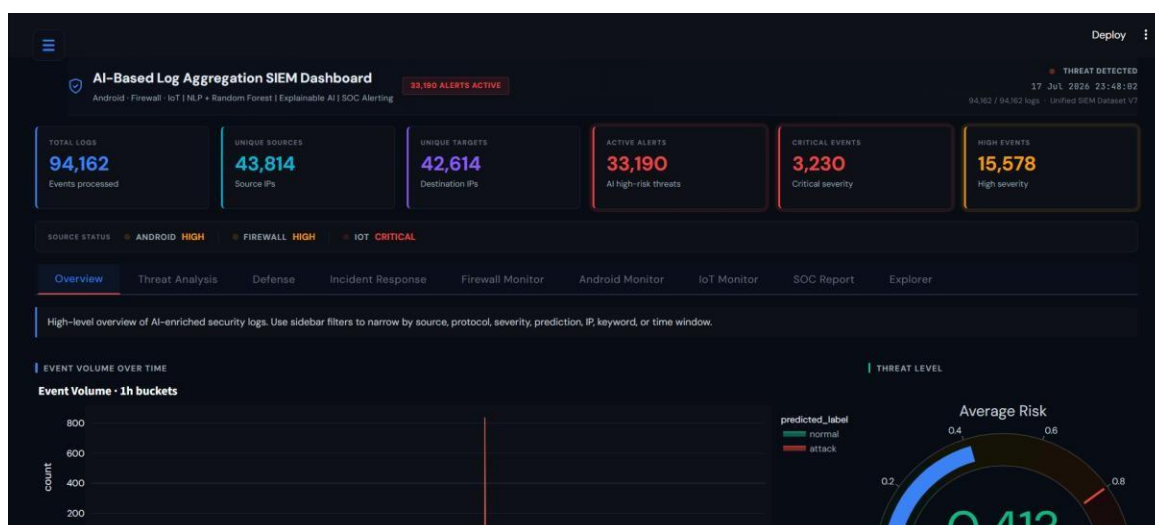


Figure 4.14 SIEM Dashboard

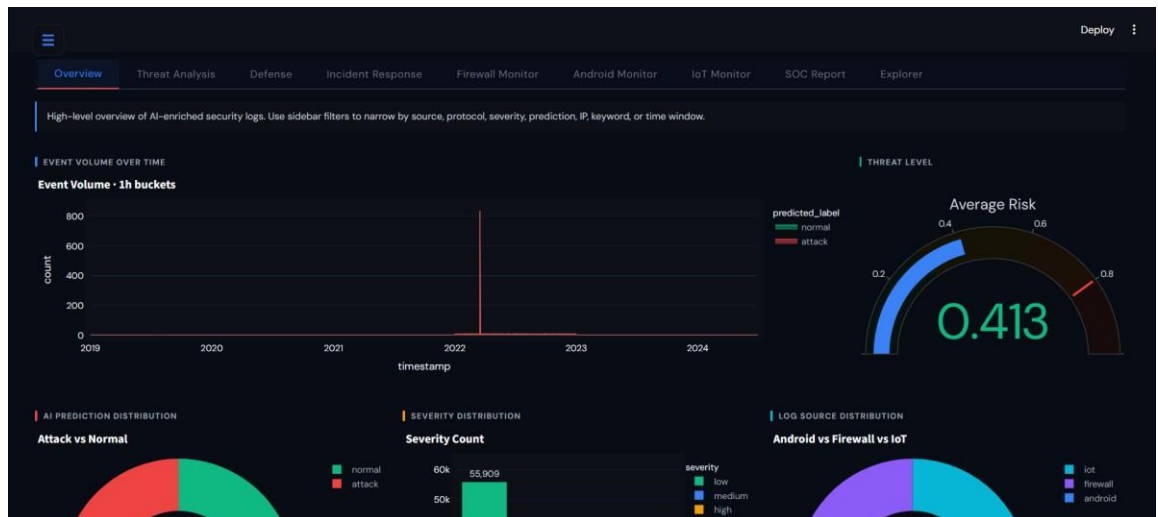


Figure 4.15 Overview Tab

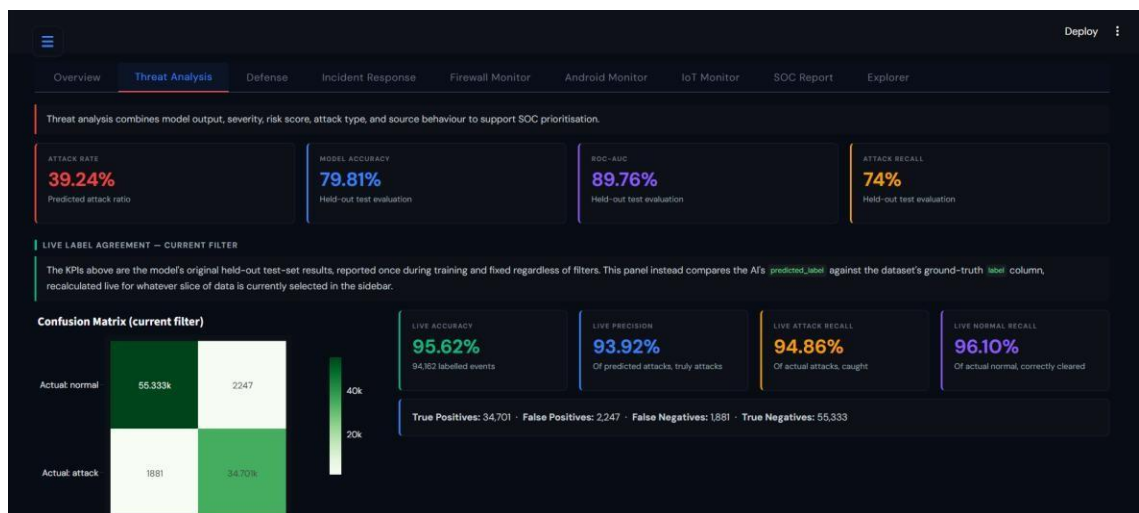


Figure 4.16 Threat Analysis Tab

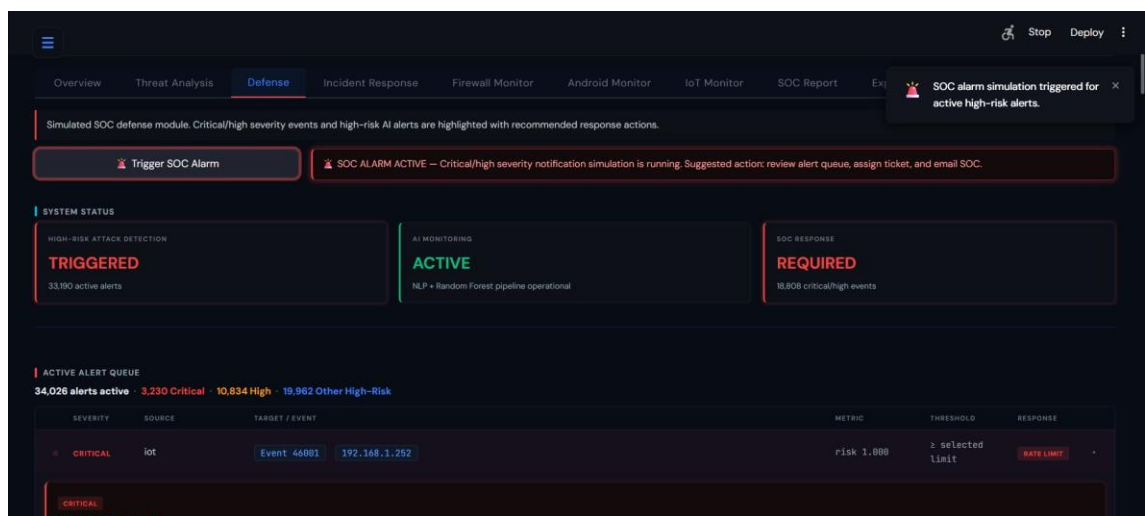


Figure 4.17 Defense Tab

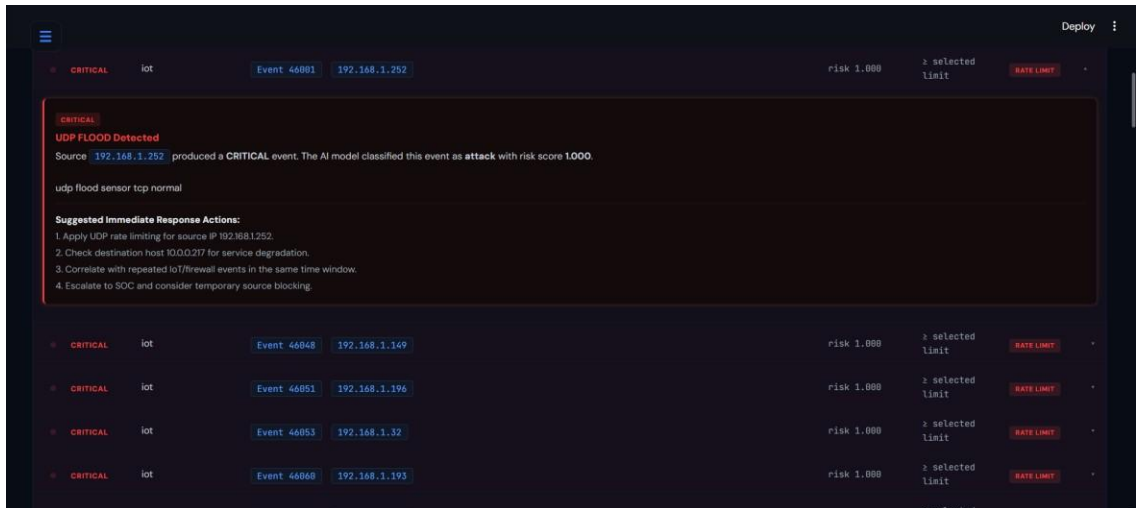


Figure 4.18 Suggested Immediate Response Actions on Critical Event (Defense Tab)

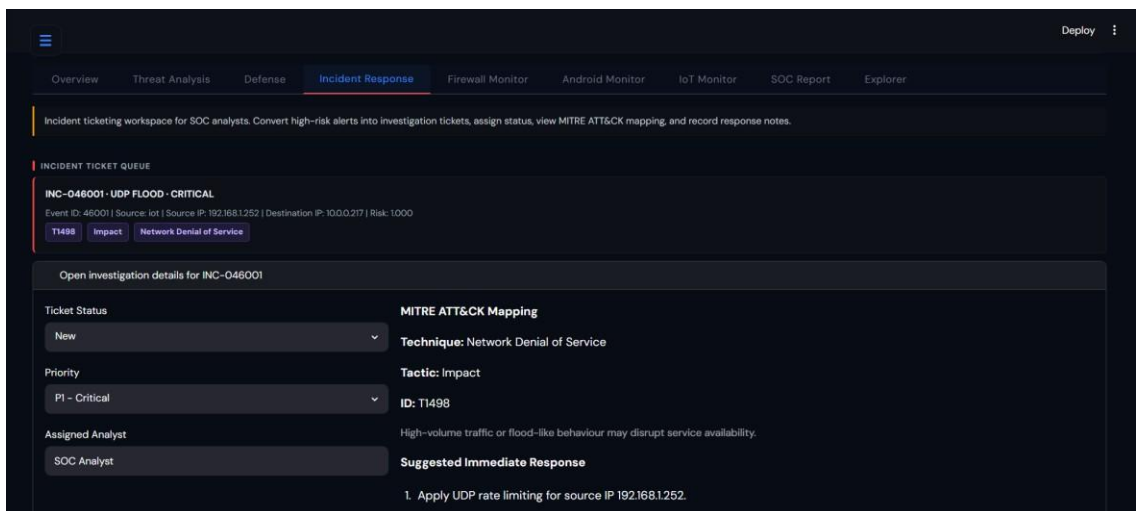


Figure 4.19 Incident Response Tab

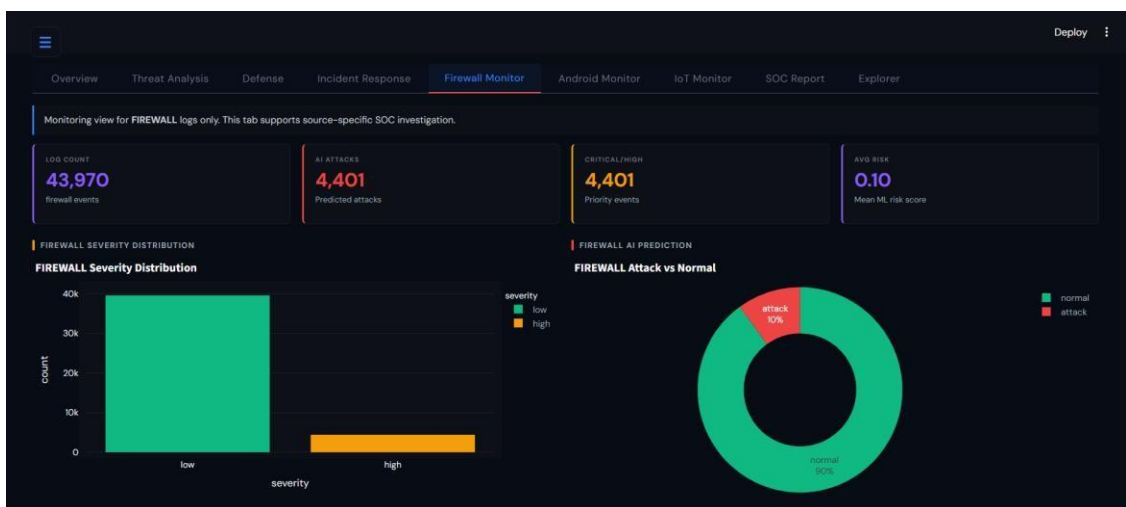


Figure 4.20 Firewall Monitoring Tab

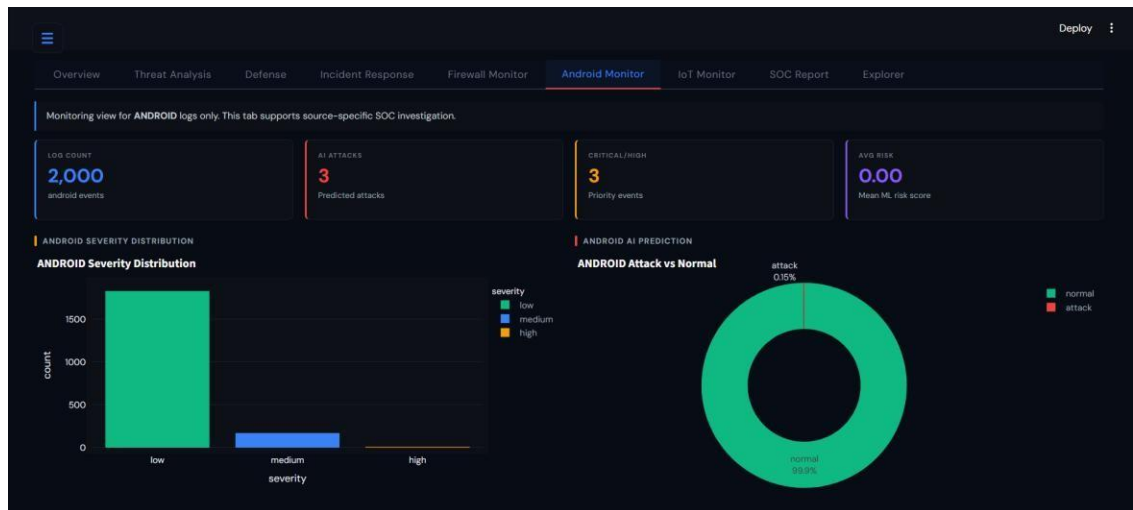


Figure 4.21 Android Monitoring Tab

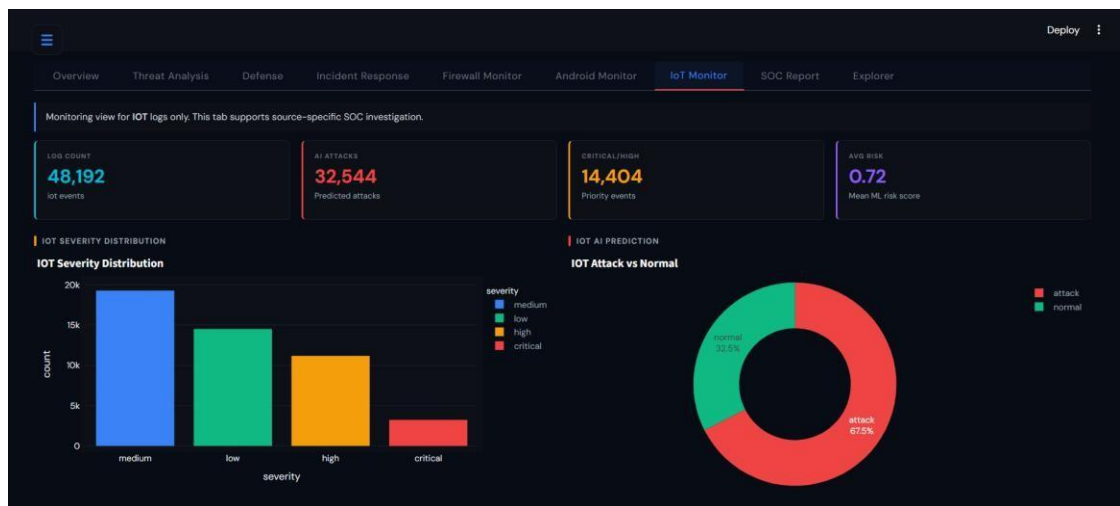


Figure 4.22 IoT Monitoring Tab

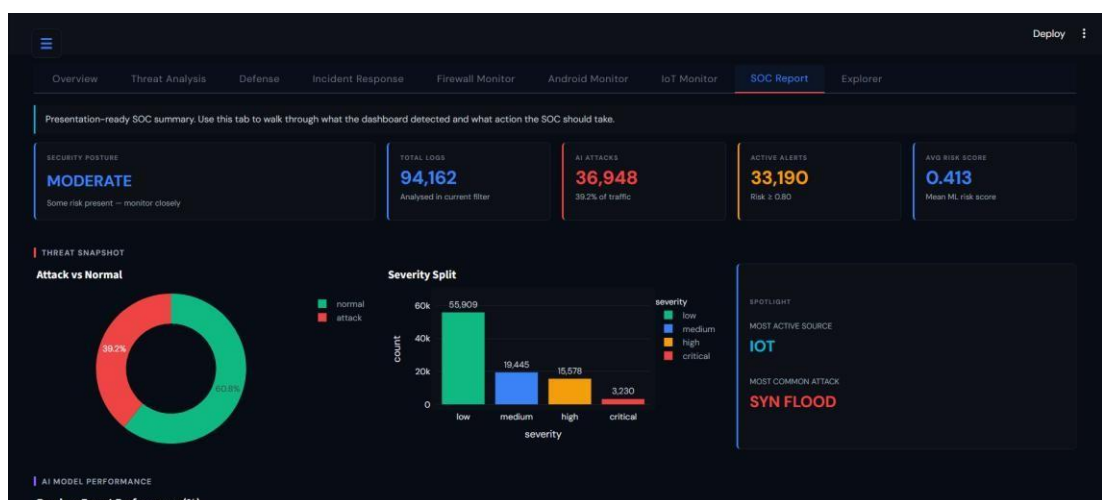


Figure 4.23 SOC Report Tab

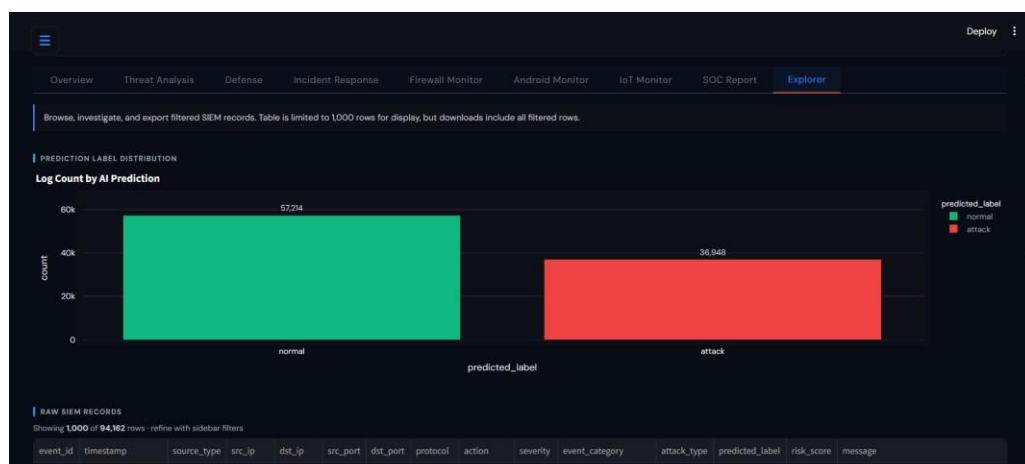


Figure 4.24 Explorer Tab

Each tab contains the same set of filter controls in the sidebar that are described in Table 4.13 and enable an analyst to filter the data to just the slice he or she is interested in for a specific investigation on 94,162 events. The functional testing scenarios T8 to T12 in Table 4.8 are used to check these controls: Combined multi-filter behaviour; Empty-result handling; Filtered data export.

Filter Control	Function
Source Type / Protocol / Severity / AI Prediction / Attack Type	Multi-select filters that narrow the dataset to specific values of each field.
High-Risk Alert Threshold slider	Sets the risk_score cut-off above which an event is counted as an active alert in the Defense tab and KPI bar (tested in T10).
Risk Score Range slider	Restricts displayed events to a chosen minimum/maximum risk_score band.
Source IP / Keyword search	Free-text search across all columns, e.g. to isolate a specific IP address or attack keyword.

Date Range picker	Restricts events to a selected time window based on the timestamp field.
-------------------	--

Table 4.13 SIEM dashboard sidebar filter controls

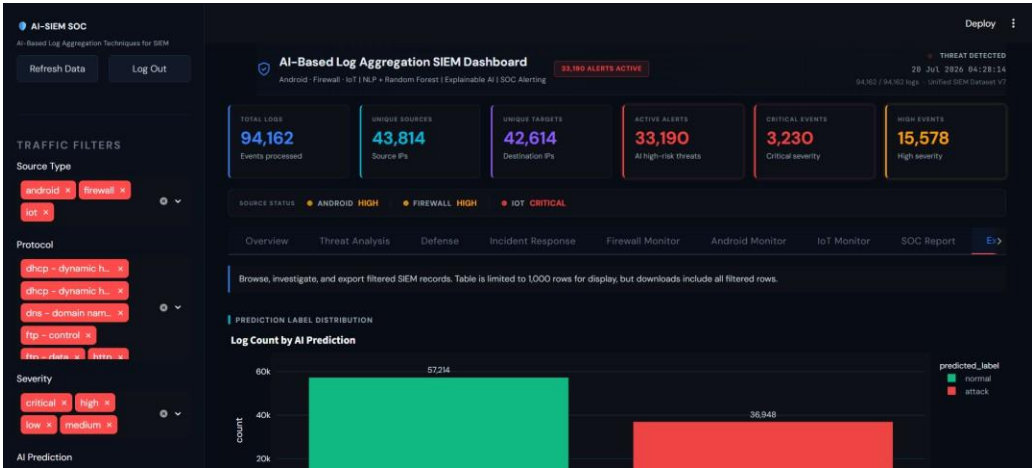


Figure 4.25 SIEM Dashboard with Sidebar Filter Controls

The per-source statistics in Section 4.2, the model evaluation results in Section 4.4.1 and the feature importance ranking in Section 4.4.2 are not one-off, static analyses but each is reproducible live as the analyst filters in the sidebar of the dashboard, as the dashboard-facing testing scenarios in Table 4.8 were designed to show.

4.5 Discussion

The results above lead to the following observations. The metric of 'agreement' to read from the 95.62% in Table 4.11 must not be interpreted as a generalisation metric: only about 80% of the rows in that table were used to train the model being scored, so that the agreement on those rows is not necessarily a reflection of generalisation performance on unseen data. The only numbers which represent a true test of the model to unseen logs are the accuracy of 79.81% and the ROC-AUC of 89.76% for the held-out test split

(Table 4.9) and these are the numbers used throughout this chapter as per the supervised evaluation procedure outlined in Section 2.4.2.

Secondly, the ranking of the feature importance is seen in Table 4.10, where only one feature, `anomaly_score`, contributes close to half of the total feature importance. This signal would be more discriminative for Android and firewall events than for IoT events, as it is based on the text and categorical data from the source events, which is expected to be more relevant for those events. This is not a theoretical concern as the mean risk scores of each source revealed in Section 4.2 offer direct empirical evidence: IoT had the highest mean score of 0.716, followed by the firewall with 0.100, and finally, Android with 0.002 - a 354-fold degree of separation. This is similar to what was mentioned in Section 4.2, where the combined data of the IoT and firewall types exceeds 97% of the data, and what was said in Section 2.6.2, that a classifier's performance on heterogeneous sources need not be assumed to be the same, just because they were trained on all of the sources together.

Third, the model has a higher recall rate for normal class (83.31%) than the attack class (74.29%), as can be seen in Table 4.9. This indicates that the model is more likely to classify a normal class attempt correctly than an attack class attempt, that is, to be correct when the true class is normal (called a false negative), than when the true class is attack (called a false positive). The significance of this asymmetry in a SIEM context is particularly important, as a missed attack gets no analyst attention whatsoever, while a false positive only adds to the triage time – making the explainability considerations discussed in Section 2.8 even more critical so that analysts can account for this bias when using the model's output. Note also that this finding implies that the class-weighting approach used in 3.4.1 was successful in ensuring that the model did not simply ignore the attack class altogether, but it is not as good as the model is in the two classes; a further tuning of the decision threshold could be considered in future research (Chapter 5).

Finally, the scenarios for functional testing in Table 4.8 all passed in tests with no unhandled errors, including tests with incomplete data (T1), with no filter results (T9), and with a split validation (T4) where the data is split

between a 1-1 range and a stratified range. Combined with the model evaluation results, this means that the system is relatively accurate and will also perform well in practice when log records are not complete or when the analyst filters are very restrictive and do not return any hits.

4.6 Findings

A single queryable dataset of 94,162 events from three log types that are structurally incompatible is successfully integrated into a single data source with the unified 27-field SIEM schema without losing the source-specific detail. In the aggregated representation, however, the text features identified by the TF-IDF, and the structured numeric and categorical features complement each other: both the tokens of the text features and the structured numeric and categorical features are present in the top-10 features of importance list (Table 4.10) and neither alone could have done as well as the combined effect in the classification results. This is also confirmation of the leaky feature reduction done during feature engineering; the model needed to explicitly remove the leakage word from the log message text and not learn the ground truth label directly, otherwise it would be cheating.

On the evaluation side, held-out accuracy of 79.81% is a more conservative and realistic validation of model performance than the 95.62% agreement seen when testing the model on all of the data on which it was partially trained, and is the number to quote when discussing the performance of this system. However, that performance is not fair across the feature space; it is highly focused on a single feature (`anomaly_score`), which comes from just one of the three log sources, and so might not be able to generalise to other log sources that have a different signal. It is also noticed that in the below-class balancing scheme, the attack-class recall rate is slightly lower than the normal-class recall rate, even after class-weighted training, showing that the attack-class balancing scheme is not a complete solution as there is still some room for tuning the thresholds in further future work.

The system was also deemed to be operationally reliable as all twelve functional test scenarios (Table 4.8) passed, showing reliability under normal use conditions, filter edge cases and simulated failure conditions. Analysing the three log sources in isolation instead of together brought to light two findings that weren't noticed when the logs were analysed together. To begin with, the mean risk score differs by a factor of over 350 between sources – 0.716 for IoT versus 0.100 for firewall and 0.002 for Android (Table 4.4) – showing that the model behaviour is not uniform but depends widely on source. Second, when tested on its own, the IoT source (94,162 events) provided an `attack_type` annotation of “no attack” (Section 4.2.3) for 2,247 events (6.9% of predicted attacks), which contradict the model's attack prediction for these same events.

CHAPTER 5

CONCLUSION

The aim of AI-Based Log Aggregation Techniques for Security Information and Event Management (SIEM) is to enhance the aggregation and analysis of diverse security logs in a Security Information and Event Management (SIEM) system. The details of the project background, literature review, methodology, system development and evaluation were presented in the previous four chapters. Hence, the final step in the project is to review the accomplishments of the project objectives, highlight the limitations of the proposed system and make recommendations for future work.

5.1 Conclusion

In this research, the design and development of an AI enabled Log Aggregation Techniques to enhance the effectiveness of Security Information and Event Management (SIEM) system was successfully achieved. The plan is to use schema standardisation and pre-processing to combine all the diverse types of security logs into a single dataset, from both the firewall environment and the Android environment or IoT environment. The system integrates NLP and Supervised Machine Learning to ensure consistent analysis of security logs across various sources, all in one platform.

The use of TF-IDF vectorisation enables log text to be represented in a meaningful way as numbers, and numerical and categorical features adds further context to the classification. These features are then merged together and fed to a Random Forest classifier that can successfully classify attack events from normal events and also provide continuous risk scores for each security event.

The experimental evaluation showed that the developed model is able to detect malicious behaviour with a held-out test accuracy of 79.81% and a

ROC-AUC of 89.76%, which is considered to be satisfactory for satisfactory classification performance. Furthermore, the feature importance analysis revealed that structured features as well as textual information were relevant to the overall prediction process, which is crucial in practical SIEM environments for explainability. Additionally, the feature importance scores enable security analysts to gain deeper insights into what drives each prediction, instead of simply a black box machine learning model.

This project also made significant efforts to reduce label leakage in the feature engineering process. The classifier learns meaningful behavioural patterns rather than memorizing obvious attack keywords since the words that are very explicit about attacks are dropped prior to the TF-IDF vectorisation step. This will help to make the trained model more reliable, and it will give a more realistic representation of how well the model will be able to classify new security events.

The current prototype is a batch processing system, but the framework provides a good starting point for future systems that use AI to assist in SIEM. Overall, the use of AI in security monitoring is a promising and innovative approach, and the various components work well together to show its potential for success in the field. In conclusion, the proposed system helps enhance the efficiency of log aggregation, security event prioritisation, and decision-making for cybersecurity analysts.

5.2 Limitation of the Project

The proposed AI Based Log Aggregation Techniques for Security Information and Event Management (SIEM) were successful in achieving the project objectives, but a number of limitations were identified during the development and testing process. The limitations must be taken into account when interpreting the findings and offer opportunities for improvement in the future.

A drawback of this project is that the ground-truth attack labels for the Android log dataset are not verified. A severity-based approach was used for relabelling Android logs for supervised machine learning. While this method can be adopted to use the dataset for model training, it is based on heuristic rules instead of "true" labels, which may introduce errors in the ground-truth data.

Another shortcoming is the classification performance of the Random Forest model. The model overall had an acceptable accuracy; however, the recall for attack events was lower than that of normal events. This means that traffic that would otherwise be considered malicious could also be considered valid traffic, thus producing false negatives and limiting the efficacy of the system to detect any security threats.

The anomaly_score feature is also heavily used in the developed model, and only available in the IoT dataset. The model might not work uniformly on all log sources because similar anomaly information is not available for the Android and firewall datasets. Consequently, the predictions for each data set can be based on different distributions of features.

Moreover, this study only compared the performance of the Random Forest classifier and did not compare it with other supervised machine-learning algorithms and simpler baseline models. As such, the benefits of the wider use of Random Forest over other classification methods were not fully explored.

The security features implemented in the SOC dashboard are also relatively basic. The dashboard is only based on one demonstration login and does not allow multiple user accounts, user management or role based access control. Thus, the authentication mechanism is well suited for prototype demonstration purposes only and does not wholly meet the security requirements of an enterprise SIEM platform.

Lastly, testing was conducted manually using a set of pre-defined functional and model evaluation scenarios. The results show that the proposed system is working as expected, but the lack of an automatic regression testing

framework could make it harder to identify any unforeseen problems or software bugs after modifications and enhancements are made in the future.

In sum, while there are some drawbacks to the proposed framework, it has showed the feasibility of embedding the use of Natural Language Processing (NLP), Feature Engineering, and Supervised Machine Learning into a consolidated log aggregation framework for SIEM applications. The restrictions found in this study offer interesting directions and suggestions for future research and system development.

5.3 Recommendation

In response to the limitations found in this study, some recommendations are made to enhance the capability, performance and practicality of the proposed framework. These recommendations can be used as possible directions for future research and system enhancement.

Going forward, work needs to be done to get verified datasets and expert-labelled datasets especially those on Android log sources to improve the quality of supervised learning, and to remove the need for heuristic label assignment. Additionally, the experiments with alternative supervised machine learning algorithms like Gradient Boosting, XGBoost, LightGBM, or Logistic Regression should be repeated to see if higher classification performance can be obtained.

Future work can also focus on ways to lessen the reliance on the `anomaly_score` feature of the model by implementing similar anomaly scores in the firewall and Android log files. This would lead to more consistent feature representation from different data sources, and to a more robust trained classifier.

A second avenue that is promising is incorporating explainable AI (XAI) methods like SHAP and LIME. The current implementation features global feature importance calculated using the Random Forest algorithm. These could be used to give detailed explanations for individual predictions,

and so help cybersecurity analysts understand why certain events are seen as attacks or normal activity.

In terms of system implementation, the framework can be extended to include streaming technologies when it comes to ingesting logs that are not necessarily batch processed but rather are real-time or near real-time. Other enhancements could involve better authentication protocols with role-based access control, multiple analysts per account and automated regression testing to maintain the integrity of the system as further enhancements are made.

In summary, these recommendations can help enhance the technical capabilities and deployment feasibility of the proposed AI-driven log aggregation system.

REFERENCES

- Abdelaziz, M. T., Radwan, A., Mamdouh, H., Saad, A. S., Abuzaid, A. S., Abdelhakeem, A. A., Zakzouk, S., Moussa, K., & Darweesh, M. S. (2024). Enhancing network threat detection with random forest-based NIDS and permutation feature importance. *Journal of Network and Systems Management*, 32(4). <https://doi.org/10.1007/s10922-024-09874-0>
- Adetunmbi, O., & Owolabi, O. (2021). Systematic review of SIEM integration for threat detection and log correlation in large networks. *International Journal of Advanced Multidisciplinary Research and Studies*.
- Ahmed, M., Seraj, R., & Islam, S. M. S. (2020). The k-means algorithm: A comprehensive survey and performance evaluation. *Electronics*, 9(8), 1295. <https://doi.org/10.3390/electronics9081295>
- Alanezi, M., & Al-Azzawi, R. M. A. (2024). AI-powered cyber threats: A systematic review. *Mesopotamian Journal of CyberSecurity*, 4(3), 166–188. <https://doi.org/10.58496/MJCS/2024/021>
- Ali, G., Shah, S., & ElAffendi, M. (2025). Enhancing cybersecurity incident response: AI-driven optimization for strengthened advanced persistent threat detection. *Results in Engineering*, 25, 10407. <https://doi.org/10.1016/j.rineng.2025.10407>
- Al-Momani, A., et al. (2024). Revolutionizing SIEM security: An innovative correlation engine design for multi-layered attack detection. *Sensors*, 24(15), 4901. <https://doi.org/10.3390/s24154901>
- Almotairi, A., Atawneh, S., Khashan, O. A., & Khafajah, N. M. (2024). Enhancing intrusion detection in IoT networks using machine learning-based feature selection and ensemble models. *Systems Science & Control Engineering*, 12(1), 2321381. <https://doi.org/10.1080/21642583.2024.2321381>
- Asudani, D. S., Nagwani, N. K., & Singh, P. (2023). Impact of word embedding models on text analytics in deep learning environments: A review. *Artificial Intelligence Review*, 56(9), 10345–10425. <https://doi.org/10.1007/s10462-022-10289-9>
- Azizi Doost, P., Sarhani Moghadam, S., Khezri, E., Basem, A., & Trik, M. (2025). A new intrusion detection method using ensemble classification and feature selection. *Scientific Reports*, 15(1), 13642. <https://doi.org/10.1038/s41598-025-98604-w>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- Chen, Z., Liu, J., Gu, W., Su, Y., & Lyu, M. R. (2021). Experience report: Deep learning-based system log analysis for anomaly detection. *arXiv*. <https://arxiv.org/abs/2107.05908>

- Chuah, E., Kalutarage, H., Tasdemir, K., Abram, A., & Maple, C. (2025). A systematic literature review of log-correlation tools for cyberattack detection and prediction in large networks. *Journal of Information Security and Applications*, 92, 104096. <https://doi.org/10.1016/j.jisa.2025.104096>
- Danish, M. (2024). Enhancing cyber security through predictive analytics: Real-time threat detection and response. *arXiv*. <https://arxiv.org/abs/2407.10864>
- Dwaraki, A., Kumary, S., & Wolf, T. (2020). Automated event identification from system logs using natural language processing. In *Proceedings of the 2020 International Conference on Computing, Networking and Communications (ICNC)* (pp. 209–215). IEEE. <https://doi.org/10.1109/ICNC47757.2020.9049750>
- Egersdoerfer, C., Zhang, D., & Dai, D. (2022). ClusterLog: Clustering logs for effective log-based anomaly detection. In *Proceedings of the IEEE/ACM Workshop on Fault Tolerance for HPC at eXtreme Scale* (pp. 1–10). IEEE.
- Fawzia Omer, A., Mohammed, H. A., Awadallah, M. A., Khan, Z., Abrar, S. U., & Shah, M. D. (2022). Big data mining using K-means and DBSCAN clustering techniques. In *Big data analytics and computational intelligence for cybersecurity* (pp. 231–246). Springer. https://doi.org/10.1007/978-3-030-93208-4_12
- Frontiers. (2025). A capability-driven automated cybersecurity monitoring and response system. *Frontiers in Computer Science*. <https://doi.org/10.3389/fcomp.2025.1692263>
- Ghazal, T. M. (2021). Performance of k-means clustering algorithm with different distance metrics. *Intelligent Automation & Soft Computing*, 30(2), 735–742. <https://doi.org/10.32604/iasc.2021.018591>
- González-Granadillo, G., González-Zarzosa, S., & Diaz, R. (2021). Security information and event management (SIEM): Analysis, trends, and usage in critical infrastructures. *Sensors*, 21(14), 4759. <https://doi.org/10.3390/s21144759>
- Hajamydeen, A. I., Suthas, S., & Abdullah, M. I. (2024). Pre-processing and event analysis for heterogeneous logs: An unsupervised approach. *Journal of Emerging Technologies and Industrial Applications*, 3(1).
- Hata, M. B. M., Darus, M. Y. B., Shafiee, M. Z. A. B., Petrus, E., & Jamian, Y. A. (2023). A log aggregation design criteria for robust SIEM in enhancing threat detection. In *Proceedings of the IEEE International Conference on Recent Advances and Innovations in Engineering* (pp. 1–6). IEEE.
- He, L., Fu, Q., Zhu, L., et al. (2024). Impact of log parsing on deep learning-based anomaly detection. *Empirical Software Engineering*, 29, Article 139. <https://doi.org/10.1007/s10664-024-10533-w>

- Heino, T. (2025). *Real-time threat detection using SIEM for industrial IoT protocols* (Master's thesis).
- Ji, Y., Liu, Y., Yao, F., et al. (2025). Adapting large language models to log analysis with interpretable domain knowledge. In *Proceedings of the ACM International Conference on Information and Knowledge Management* (pp. 1135–1144). ACM.
- Kalakoti, R., Vaarandi, R., Bahsi, H., & Nömm, S. (2025). Evaluating explainable AI for deep learning-based network intrusion detection system alert classification. In *Proceedings of the 11th International Conference on Information Systems Security and Privacy (ICISSP 2025)*. <https://arxiv.org/abs/2506.07882>
- Khan, Z. A., Shin, D., Bianculli, D., et al. (2024). Impact of log parsing on deep learning-based anomaly detection. *Empirical Software Engineering*, 29, Article 139. <https://doi.org/10.1007/s10664-024-10533-w>
- Khayat, M., et al. (2025). Empowering security operation centers with AI and ML: A systematic literature review. *IEEE Access*, 13, 19173–19195. <https://doi.org/10.1109/ACCESS.2025.10850912>
- Kumar, S., et al. (2024). Big data-driven SIEM enhanced by artificial intelligence. *Educational Administration: Theory and Practice*.
- Lu, J., & Wu, C. (2024). TPLogAD: Unsupervised log anomaly detection based on event templates and key parameters. *arXiv*.
- Manzoor, A., et al. (2025). A security information and event management (SIEM) implementation for small businesses. *Theseus Research Repository*. <https://www.theseus.fi/handle/10024/895242>
- Menges, F., et al. (2021). Towards GDPR-compliant data processing in modern SIEM systems. *Computers & Security*, 103, 102165. <https://doi.org/10.1016/j.cose.2020.102165>
- Mokalled, H., et al. (2020). Guidelines to adopt an applicable SIEM solution. *Journal of Information Security*, 11(1).
- Ndibe, O. S. (2025). AI-driven forensic systems for real-time anomaly detection and threat mitigation. *International Journal of Research Publication and Reviews*, 6(5), 389–411.
- Nurusheva, A. M., et al. (2024). Machine learning algorithms in SIEM systems. *Bulletin of LN Gumilyov Eurasian National University*, 148(3), 6–17.
- Palma, G., et al. (2025). Leveraging large language models for scalable cybersecurity log analysis. *Journal of Cybersecurity and Privacy*, 5(3), 55. <https://doi.org/10.3390/jcp5030055>
- Piet, J., et al. (2025). Semantic-aware parsing for security logs. *arXiv*. <https://arxiv.org/abs/2506.17512>

- Pospieszny, P., et al. (2025). ADALog: Adaptive unsupervised anomaly detection in logs. *arXiv*.
- PwC Germany. (2024). *Evolving SIEM: From traditional log management to AI-driven security operations* (White paper).
- Qin, Q., et al. (2025). Preprocessing is all you need: Boosting log parser performance. In *Proceedings of SANER*. IEEE.
- Ravipati, H. (2025). Neuromorphic mimicry attacks exploiting brain-inspired computing. *arXiv*. <https://arxiv.org/abs/2505.17094>
- Saleem Raja, A., Vinodini, R., & Kavitha, A. (2021). Lexical features based malicious URL detection using machine learning techniques. *Materials Today: Proceedings*, 47, 163-166. <https://doi.org/10.1016/j.matpr.2021.04.041>
- Sarker, I. H. (2021). CyberLearning: Machine learning security modeling. *Internet of Things*, 14, 100393. <https://doi.org/10.1016/j.iot.2021.100393>
- Sarker, I. H. (2023). Machine learning for intelligent cybersecurity automation. *Annals of Data Science*, 10(6), 1473–1498. <https://doi.org/10.1007/s40745-022-00424-8>
- Sheeraz, M., et al. (2023). Effective security monitoring using efficient SIEM architecture. *Human-Centric Computing and Information Sciences*, 13(17).
- Studiawan, H., & Sohel, F. (2020). Performance evaluation of anomaly detection in imbalanced log data. In *Proceedings of WorldS4* (pp. 239–246). IEEE.
- Tendikov, N., et al. (2024). SIEM data acquisition and analysis with machine learning principles. *Results in Engineering*, 22, 102254. <https://doi.org/10.1016/j.rineng.2024.102254>
- Uetz, R., et al. (2023). Detecting evasions of SIEM rules in enterprise networks.
- Waliullah, M., et al. (2025). Cybersecurity threats and digital banking adoption: A systematic review. *arXiv*. <https://arxiv.org/abs/2503.22710>
- Zhang, S., et al. (2024). End-to-end AutoML for unsupervised log anomaly detection. In *Proceedings of ASE '24*. ACM.
- Zhao, Y., et al. (2025). Using large language models for template detection from security event logs. *International Journal of Information Security*, 24, Article 104. <https://doi.org/10.1007/s10207-025-01018-y>